



Automated Tooth Segmentation & Anatomical Landmark Detection Deep
Learning Pipeline

Project No: CCDS25-0544

Name: Nathaniel Lo Tzin Ye

Matriculation Number: U2220882C

College of Computing & Data Science

Academic Year 2025/2026

Nanyang Technological University

CCDS25-0544:

Automated Tooth Segmentation & Anatomical
Landmark Detection Deep Learning Pipeline

Submitted in Partial Fulfilment of the Requirements for the Degree of Bachelor of
Engineering in Computer Engineering of the Nanyang Technological University

by

Nathaniel Lo Tzin Ye

College of Computing & Data Science

Academic Year 2025/2026

Abstract

Three-dimensional intraoral scanning has become a standard clinical tool in modern dentistry, providing high-resolution digital replicas of a patient's dentition with sub-millimetre precision. However, the manual analysis of these scans, including the identification of individual teeth and localisation of anatomical dental landmarks, remains a time-consuming and labour-intensive matter that is often subject to examiner variability. This project presents an end-to-end deep learning pipeline that automates both tasks from a single raw jaw scan, requiring no manual interaction or intervention beyond the initial scan acquisition.

The pipeline operates in two sequential stages. In the first stage, a point cloud segmentation network assigns each mesh vertex an FDI tooth label, distinguishing up to 16 individual teeth from gingival tissue across a complete upper and lower jaw scan. Subsequently, a StratifiedTransformer-based landmark detection network localises each segmented tooth with 6 predefined anatomical landmark classes, which comprise the mesial, distal, cusp, inner point, outer point and facial point landmark points. A custom coordinate-space bridge resolves the coordinate-space mismatch between the two independently trained networks, enabling them to function as a unified and coherent pipeline..

Acknowledgements

I would like to offer my sincerest gratitude for the opportunity to contribute to this final year project, which has allowed me to explore deep learning techniques within the medical industry, especially in orthodontics. Working on this project allowed me to gain invaluable experiences in the area.

I would like to express my utmost gratitude to Professor Zheng Jianmin for providing this opportunity for me to work on this project. His support and responsiveness throughout the project have been instrumental in the success of this project.

Last but not least, I would like to express my gratitude to my family and friends who supported my journey in CCDS of NTU. Their unwavering support has been crucial in enabling me to pursue my academic goals here.

Table of Contents

Abstract	3
Acknowledgements	4
Table of Contents	5
List of Figures	7
List of Tables	9
Chapter 1 Introduction	10
1.1 Background	10
1.2 Problem Statement & Project Objectives	11
1.2.1 Tooth Instance Segmentation	11
1.2.2 Anatomical Landmark Detection	13
1.3 Report Organisation	14
Chapter 2: Literature Review	16
2.1 Deep Learning for 3D Point Cloud Processing	17
2.2 Tooth Segmentation Methods	18
2.3 Anatomical Landmark Detection in 3D Medical Imaging	19
Chapter 3 Dataset	20
3.1 Teeth3DS dataset	20
3.1.1 Teeth3DS dataset statistics	21
3.1.2. FDI World Dental Federation Notation System	21
3.1.3 Mesh File Format (.obj)	24
3.1.4 Segmentation Annotation Format (.json)	25
3.2 3DTeethLand Landmark Dataset	26
3.2.1 Landmark Annotation Format (_kpt.json)	28
Chapter 4 Methodology	30
4.1 System Architecture Overview	30
4.2 Stage 1: Tooth Instance Segmentation	33
4.2.1 Input Parsing & Normalisation	33
4.2.2. Farther Point Sampling (FPS)	34
4.2.3 Point Transformer Encoder-Decoder Segmentation Network	35
4.2.4 Boundary Aware Deep Learning Network	39
4.2.5 FDI Label Assignment	41
4.3 Stage 2: Coordinate Space Bridge	42
4.3.1 Motivation	42
4.3.2 Scale Factor Derivation & Centroid Conversion	42

4.3.3 Full Jaw Z-Score Normalisation	43
4.3.4 Per Tooth Crop Extraction with KDTree	44
4.3.5 PointTensor Construction with Feature Vector	45
4.4 Stage 3: Landmark Detection	47
4.4.1 Overview: Stratified Transformer with Six Output Heads	47
4.4.2 KPConv Point Embedding	48
4.4.2 Stratified Transformer Encoder with Windowed Attention	49
4.4.3 Stratified Transformer Decoder with Six Independent Prediction Heads	51
4.5 Post-Processing and Landmark Clustering	53
4.5.1 Candidate Landmark Extraction	53
4.5.2 DBSCAN Landmark Clustering	55
4.6 Output Visualisation	56
Chapter 5 Results & Evaluation	58
5.1 Evaluation Metrics	58
5.1.1 Tooth Segmentation: Teeth Localisation Accuracy (TLA)	58
5.1.2 Tooth Segmentation: Teeth Segmentation Accuracy (TSA)	58
5.1.3 Tooth Segmentation: Teeth Identification Rate (TIR)	58
5.1.4 Tooth Segmentation: Combined Score	59
5.1.5 Landmark Detection: Mean Average Precision (mAP)	59
5.1.6 Landmark Detection: Mean Average Recall (mAR)	59
5.2 Qualitative Results	60
5.3 Quantitative Results	61
5.4 Evaluation Scores Discussion	62
5.5 Visualisation Files Output	63
5.6 Visualisation Files Output Discussion	68
Chapter 6 Conclusion & Future Works	71
6.1 Conclusion	71
6.2 Future Works	72
References	73
Appendix	76
Hyperparameters	76

List of Figures

Figure 1: Dental quadrant with respect to dental midline [8]

Figure 2: FDI Notation Quick Reference Diagram [11]

Figure 3: Snippet of mesh file format from Teeth3DS (.obj)

Figure 4: Snippet of content within landmark annotation JSON

Figure 5: Architecture Flow Diagram

Figure 6: Visualised step-by-step pipeline architecture

Figure 7: Point Transformer Encoder - Decoder Visualisation

Figure 8: Output / Creation of octahedra .ply file for...

Figure 9: File Directory of sample 01F4JV8X_upper

Figure 10: 3D display of 14 segmented teeth with colour coding...

Figure 11: Landmark labelled by colour on 01F4JV8X_upper

Figure 12: Evaluation Metrics score in terminal output

Figure 13: Original 01F4JV8X_upper.obj mesh scan

Figure 14: 01F4JV8X_upper.obj mesh scan with individual coloured teeth...

Figure 15: 01F4JV8X_upper_colored.ply with all landmark labels displayed on...

Figure 16: 01F4JV8X_upper_Cusp.ply displaying blue octahedron indicating Cusp landmarks...

Figure 17: 01F4JV8X_upper_Distal.ply displaying green octahedron indicating Distal landmarks...

Figure 18: 01F4JV8X_upper_Mesial.ply displaying red octahedron indicating Mesial landmarks...

Figure 19: 01F4JV8X_upper_InnerPoint.ply displaying yellow octahedron indicating inner...

Figure 20: 01F4JV8X_upper_OuterPoint.ply displaying cyan octahedron indicating outer point...

Figure 21: 01F4JV8X_upper_FacialPoint.ply displaying purple octahedron indicating facial point...

List of Tables

Table 1: Anatomical description of the different recognised landmark classes

Table 2: Teeth3DS dataset statistics

Table 3: FDI Notation Quick Reference Table

Table 4: Segmentation Annotation json file field description

Table 5: Expected count of landmark class per tooth

Table 6: Description of fields in the landmark annotation JSON

Table 7: Point Transformer Encoder Layer Specifications

Table 8: Point Transformer Decoder Layer Specifications

Table 9: Boundary Refinement Encoder Specifications

Table 10: Boundary Refinement Decoder Specifications

Table 11: PointTensor 9-channel feature vector contents & specifications

Table 12: Stratified Transformer Encoder Downsampling Process

Table 13: Prediction Heads and corresponding output channel dimensions

Table 14: Landmark position 4-channel prediction encoding

Table 15: Output files

Table 16: Evaluation Metric Scores

Chapter 1 Introduction

1.1 Background

In modern orthodontics, oral health assessment is traditionally grounded in the evaluation of occlusal alignment [1], jaw growth patterns and craniofacial morphology [2]. The digitisation of dental clinic workflows that conduct such assessments has developed rapidly, with real-world implementations very much prevalent and deeply embedded in orthodontic practice worldwide. Where physical plaster study models and hand-traced cephalometric radiographs once defined the standard of orthodontic healthcare, intraoral optical scanners are now routinely used to capture sub-millimetre, three-dimensional surface models of patient dentition within minutes of chair time.

Current state-of-the-art intraoral 3D scanners such as Primescan (Dentsply), TRIOS 3 (3Shape) and iTero Element 2 Plus [11] are able to produce high-quality 3D models with dimensional accuracy down to the micrometre, allowing massive adoption in clinics for better orthodontic assessments. This transition to incorporating fully digital workflows has transformed orthodontic practices, restorative dentistry and oral surgery. Digital models can be shared electronically between specialists and used as direct input to computer-aided design and manufacturing (CAD / CAM) systems for major dental procedures like aligner and braces production, crown fabrication and surgical guides. These capabilities dramatically allow clinicians today to simulate treatment plans, predict outcomes, and better manage patient expectations, which simultaneously reducing the uncertainties inherent in traditional workflows.

Despite the precision and convenience of such modern intraoral scanners, the downstream analysis of the resulting digital models is placed substantially on experienced and professional clinicians. To put into perspective, a qualified orthodontist performing an initial patient assessment must undertake a structured sequence of measurements and planning steps that depend critically on the accurate identification of individual teeth and the precise localisation of specific anatomical reference points on each tooth surface. When performed manually, both tasks are time-consuming, expertise-dependent, and prone to inter-examiner variability.

1.2 Problem Statement & Project Objectives

The project addresses two core technical sub-problems that will be solved sequentially through a designated pipeline.

1.2.1 Tooth Instance Segmentation

Given a 3D triangular surface mesh which represents the scanned upper or lower jaw, the objective of this task is to assign a semantic label according to the Fédération Dentaire Internationale (FDI) notation standards to the mesh vertices of the scan that indicate the important points of the tooth's structure. The notation system follows the unique two-digit tooth notation standards of the FDI [10], which will be further explained in Chapter 3 detailing our dataset. The notation is needed for the model to properly label and identify individual teeth by their FDI tooth number, while also segmenting the portions associated with the gingival background.

The problem is a semantic segmentation task that requires the tooth class for each mesh vertex to be determined accurately, and an instance segmentation task that requires distinguishing teeth in different quadrants of the jaw. However, concurrent factors make tooth segmentation a fundamentally challenging task. A fundamental difficulty is the severe class imbalance between the gingival background and individual tooth classes, as the gingiva occupies a disproportionately large share of mesh vertices relative to any single tooth label, causing models trained without explicit compensation to overfit toward the majority class [4]. Further, the ambiguity of inner-tooth boundaries, particularly in regions of tight proximal contact points between teeth, where the scanned surface presents virtually no geometric discontinuity between neighbouring crowns, forces the model to infer boundaries from broader contextual shape cues rather than local geometry [4] [5]. The high degree of intra-class geometric similarity, whereby teeth of the same type on opposite sides of the arch are near-mirror images of one another, further complicates correct FDI label assignment without substantial global arch-level context. [13]

Patient-level variability also introduces additional difficulty, as dentitions are considerably different across individuals due to factors like missing teeth, disarranged or rotated teeth and incomplete scans. To put into context, only approximately 5% of scans in the Teeth3DS dataset show wisdom teeth present during scans, making these tooth classes underrepresented

during training [6] [7]. The orthodontic composition of the dataset further exacerbates this variability, as some patients suffer from dental anomalies, asymmetric arches and orthodontic appliances such as braces that occlude and distort the visible crown geometry during scanning [16]. Finally, downsampling of the final sample risks discarding geometrically informative vertices at tooth boundaries, which reduces segmentation precision at margins between the crown and gingival areas [13] [18].

Furthermore, semantic segmentation methods that predict a single tooth class for all teeth cannot support landmark detection for individual teeth, since landmarks are associated with a specific FDI-labelled tooth. Meanwhile, instance segmentation is substantially harder than semantic segmentation as it requires a discriminative embedding for each point and points of the same tooth to cluster together in the feature space, and points of different tooth instances to be separated. State-of-the-art instance segmentation approaches for 3D dental scans combine semantic label prediction with an explicit grouping or boundary detection stage to produce clean per-tooth instance masks.

1.2.2 Anatomical Landmark Detection

Due to the limitations of the tooth instance segmentation methods, identifying and labelling the areas of different teeth with the correct landmark class becomes a unique challenge itself. Given a point cloud representing a single segmented tooth, six landmark classes need to be localised to sub-millimetre spatial accuracy.

Landmark Class	Anatomical Description
Mesial	Contact point on the proximal surface facing the dental midline.
Distal	Contact point on the proximal surface facing away from the midline.
Cusp	Tip of a buccal or lingual cusp, or the incisal edge of an anterior tooth.
Inner Point	The deepest point of the central fossa or the most prominent marginal ridge.
Outer Point	The most convex point of the buccal or labial surface.
Facial Point	Mid-facial surface reference point used in bracket placement planning.

Table 1: Anatomical description of the different recognised landmark classes

Landmark detection on intraoral 3D scans presents several challenges that distinguish it from standard 2D detection tasks. The number of landmark instances that vary per tooth and per class requires the detection framework to produce a variable number of outputs rather than a fixed set, a property that challenges participants addressed by designing dedicated variable-output decoder branches specifically for cusp prediction [19]. Beyond cardinality, certain landmark classes, such as the facial axis point, lack a sharp, unambiguous geometric correlate on the tooth surface and are instead defined by clinical convention, introducing irreducible annotation uncertainty that propagates directly into the learning target and increases inter-annotator variability [7]. Compounding both of these issues is the requirement

for sub-millimetre spatial accuracy: landmarks must be localised in 3D space to within approximately 1 mm over a per-tooth point cloud that spans only 20–40 mm in extent, demanding fine-grained spatial reasoning that is practically challenging due to the large shape variance across tooth types, the irregular and non-uniform sampling of point clouds produced by intraoral scanners, and the limited availability of annotated training data [7], [13].

1.3 Report Organisation

The final report is structured in six chapter, where each chapter will address a certain aspect of the automated tooth segmentation and anatomical landmark detection pipeline.

Chapter 1: Introduction establishes the introduction to the topic of deep learning applications within the dental clinic context and the clinical motivation for such a project. The chapter outlines the shift towards digital dental workflows as well as the limitations of current systems and their successful implementations. The two core technical problems of tooth instance segmentation and anatomical landmark detection are formally defined, along with key challenges associated with each problem statement.

Chapter 2: Literature Review will gather relevant works and research in three relevant areas to the topic, namely deep learning architectures for 3D point cloud processing, existing tooth segmentation methods, and anatomical landmark detection in 3D medical imaging. The chapter reviews real-world segmentation and landmark annotation challenges from International Conference on Medical Image Computing and Computer Assisted Intervention to address the study within the field and their top-performing solutions are reviewed to contextualise the design decisions made in this project.

Chapter 3: Dataset will present the two datasets used in this project, which are the Teeth3DS dataset for tooth segmentation from the MICCAI2022 Challenge, and the 3DTeethLand dataset for landmark detection from the MICCAI2024 Challenge. Relevant information to understand the datasets are also included within this chapter, such as the FDI notation system, mesh file formats, segmentation annotation formats, and landmark annotation formats.

Chapter 4: Methodology details the full deep learning pipeline architecture implemented through the project. Stage 1 of the pipeline covers the tooth instance segmentation task, which utilises a Point Transformer encoder-decoder network with furthest point sampling and teeth mesh boundary refinement techniques. Stage 2 introduces the transition bridge that

connects the two tasks by aligning their coordinate space to match each others. Stage 3 of the pipeline manages the landmark detection task using a Stratified Transformer with six independent prediction heads. Post-processing of the output from the pipeline leverages techniques like DBSCAN clustering to finalise the results and precise landmarks, which is correspondingly designed with a proper visualisation of the final output to complete the entire pipeline.

Chapter 5: Results and Evaluation presents the qualitative and quantitative evaluation of the pipeline on a sample scan. The evaluation metrics for both segmentation and landmark detection tasks are defined, and the pipeline's performance according these metrics are reported clearly.

Chapter 6: Conclusion and Future Works summarises the entire project contribution and reflects on the pipeline's capabilities but also on its limitations. The report closes with plans and proposals to further improve the deep learning pipeline.

Chapter 2: Literature Review

In this chapter, a literature review is done to discover past, present and future advancements and implementations of intraoral scanners with 3D teeth models and their use under clinical settings. The literature review will discover the past problems which brought about the ability to use such scanners and the current real-world implementation of the scanners and their subsequent models. Furthermore, future implementations and limitations of the current clinical workflows will also be presented, along with relevant works in dental segmentation and landmark detection that utilise the infrastructure above.

The use of three-dimensional digital models in healthcare, especially dentistry, has grown significantly since the rapid commercial introduction of intraoral scanners in the 2000s. Systems such as the iTero Element, 3Shape TRIOS, and Dentsply Primescan reconstruct the full dental arch as a triangular surface mesh through a simple scan of the jaw, replacing physical plaster study models that orthodontists previously relied upon for treatment planning [21].

The digital model serves as the primary input for the two relevant tasks of this project that will be handled with deep learning techniques. Firstly, clinicians need to identify and label the teeth within the mesh model accordingly, following the FDI two-digit notation system, which constitutes our first task. Following that, clinicians need to place anatomical landmark markers at specific locations on each tooth surface, which will be used as reference points to fulfil certain clinical needs. This step constitutes our second task of anatomical landmark detection through supervised learning. When performed manually, both tasks present a time-consuming and expertise-dependent challenge.

Prior to the widespread adoption of intraoral surface scanning, computational dental analysis focused primarily on two-dimensional panoramic radiographs and cone beam computed tomography (CBCT), relying heavily on two-dimensional projections. However, these modalities tend to compress and obscure the finer anatomical surface details required for accurate segmentation and landmark localisation. Intraoral optical surface scanning, therefore, became the modality of choice for orthodontic measurements, shifting research effort towards utilising 3D mesh.

To address the limited availability of publicly accessible annotated data or databases for 3D dental analysis, the Teeth3DS benchmark dataset was released in conjunction with the 3DTeethSeg'22 challenge at MICCAI 2022 [12]. The dataset comprises 1800 intraoral scans from 900 patients, with per-vertex FDI tooth labels and instance annotations validated by orthodontists and dental surgeons. The subsequent 3DTeethLand challenge at MICCAI 2024 extended this benchmark with anatomical landmark annotations for a subset of 340 scans, providing the first publicly available dataset for 3D tooth landmark detection [19]. Together, these two benchmarks have established the standardised evaluation protocols and data splits used throughout the present work.

2.1 Deep Learning for 3D Point Cloud Processing

Intraoral scanners present the teeth model scans in the form of a 3D mesh, which is made up of its constituent point clouds within the space. Unlike images, point clouds are unordered and irregularly sampled, making standard convolutional applications incompatible. Nevertheless, the development and adoption of new deep learning architectures have addressed such issues.

PointNet [24] is a deep learning architecture that processes raw 3D point clouds directly without converting them into other more conventional representations. It achieves this by applying a shared multi-layer perception (MLP) independently to each point and aggregating features via global max pooling. While PointNet is effective for shape classification, the global pooling discards localised geometric structure.

PointNet++ [25] addresses that by introducing a hierarchical Set Abstraction (SA) module in three stages. Firstly, farthest point sampling (FPS) will select a set of centroids that are maximally spread across the point cloud. Then, a ball query will group the k -nearest points within a specified radius from the selected centroids in the previous step. Finally, a local PointNet will process each group to produce a feature vector for the centroid. By stacking SA layers that repeat the steps with increasing radii and decreasing centroid counts, PointNet++ can encode the geometry of differing neighbourhood scopes, understanding local and global contexts.

SA1: 10,000 pts $\rightarrow$ 2,048 centroids, radius 0.05, features capture fine curvature

SA2: 2,048 pts $\rightarrow$ 512 centroids, radius 0.10, features capture crown shape

SA3: 512 pts $\rightarrow$ 128 centroids, radius 0.20, features capture whole tooth

SA4: 128 pts $\rightarrow$ 32 centroids, global, features capture arch context

Transformer-based architectures have also been introduced for such implementations. The Point Transformer [26] applies self-attention within local k-nearest-neighbour point cloud groups with attention weights modulated by positional encodings. The Stratified Transformer [27] scales the Point Transformer to accommodate larger point clouds by partitioning inputs into non-overlapping spatial windows at each resolution level, and self-attention is computed within each window. This allowed the Stratified Transformer to be more scalable to point clouds with tens of thousands of points.

2.2 Tooth Segmentation Methods

Tooth Segmentation from 3D intraoral scans has evolved substantially from early handcrafted geometric approaches to fully learned architectures. Traditional methods provided a substantial foundation for the field, but manual intervention was required, which deterred fully automatic implementations.

The 3DTeethSeg'22 MICCAI challenge [12] provided a competitive evaluation of automated tooth segmentation methods based on a standardised benchmark, evaluating algorithms according to teeth localisation accuracy (TLA), teeth segmentation accuracy (TSA), and teeth identification rate (TIR). The top-performing CGIP method by Lim et al. achieved an overall score of 0.9539 using a dual-module architecture: a Point Grouping Module based on a Point Transformer backbone predicted per-tooth semantic labels, and a Tooth Cropping Module refined the instance boundaries using a tooth-gingiva mask. The Chamfers team (van Nistelrooij et al.) used a Stratified Transformer encoder-decoder architecture for centroid prediction, followed by a cascaded two-stage tooth segmentation, achieving a competitive segmentation accuracy (TSA 0.8886). The IGIP team utilised a multi-stage pipeline that combined teeth-gingiva separation, PointNet++-based centroid prediction with offset regression and patch-based individual tooth segmentation with shape and position for

classification, achieving the highest teeth identification rate (TIR 0.9289). Overall, the challenge results established that the inter-tooth boundary region and handling of missing or partially erupted teeth were a primary source of segmentation error.

2.3 Anatomical Landmark Detection in 3D Medical Imaging

Automated detection of anatomical landmarks on 3D dental meshes is a comparatively less studied problem than segmentation. Early approaches relied more heavily on handcrafted geometric features. Woodsend et al. [23] proposed a sequential pipeline for landmark detection on digital dental models based on orientation estimation, local maxima identification, and rule-based filtering, demonstrating the feasibility of automated landmark identification but with limited generalisability across diverse dental morphologies. Triarjo et al. extended this by combining deep learning for tooth segmentation with a weighted mesh point transformation for landmark localisation, retaining the dependency on handcrafted geometric descriptors at the final stage.

The 3DTeethLand challenge at MICCAI 2024 [19] also provided a systematic evaluation of 3D dental landmark detection methods at scale, introducing a dataset of 340 annotated intraoral scans and evaluating six algorithms using mean average precision (mAP) and mean average recall (mAR) across four landmark categories. The top-performing Radboud method (van Nistelrooij et al.) achieved a rank score of 0.9172, an overall mAP of 0.785, and an mAR of 0.656. Its landmark detection stage used one shared encoder and five separate decoders that predicted per-point distance and offset fields, with DBSCAN clustering applied to the resulting proposals to recover final landmark positions. Critically, a separate sixth decoder handled cusp landmarks specifically to accommodate their variable count per tooth. The second-ranked ChohoTech method demonstrated that competitive accuracy (mAP 0.775) could be achieved without an explicit segmentation stage using a DGCNN-based dual-branch network predicting probability heatmaps and offset vectors jointly. The challenge results confirmed that the distance field formulation with post-hoc clustering is currently the dominant paradigm for 3D dental landmark detection, and that handling the variable cardinality of cusp landmarks requires explicit architectural accommodation, either through dedicated decoder branches or variable-output detection heads [19].

Chapter 3 Dataset

3.1 Teeth3DS dataset

The Teeth3DS dataset [12] was released as part of the 3DTeethSeg’22 MICCAI Challenge and is the largest publicly available benchmark for intraoral scan segmentations. It comprises 1800 upper and lower jaw scans from 900 patients, captured by experienced orthodontists with more than 5 years of professional experience using conventional intraoral scanners from clinic primarily located in France and Belgium. The dataset was assembled from anonymised patients requiring orthodontic or prosthetic treatments, with balanced contribution from each group. The patient cohort reflects a realistic orthodontic demographic of 50% male and 50% female, where among them, approximately 70% are under 16 years-old, approximately 27% were patients aged between 16 and 59 years old, and approximately 3% were over 60 years old. This reflects the real distribution of orthodontics patient rather than the general population age distribution. A younger demographic is observed, predictably due to orthodontic therapy and practices like braces and aligners are initiated early during adolescence.

Each scan in the Teeth3DS dataset consists of a triangular surface mesh in OBJ format with per-vertex colour information (which is stored as values v, x, y, z, r, g, b in the vertex list), a JSON annotation file containing an array of per-vertex integer representing the teeth FDI labels (0 for gingiva, 11-48 for individual teeth), and a per-vertex instance index array that distinguishes instances of the same tooth type in different quadrants.

3.1.1 Teeth3DS dataset statistics

Property	Value
Total Scans	1800
Patients	900
Jaws scans per patient	2 (upper jaw & lower jaw)
Mean vertices per scan	100000
FDI labels present	11 - 48, with up to 16 per scan
Gingiva label	0
Format	OBJ and JSON
Train Split	1200 scans (600 patients)
Test Split	600 scans (300 patients)

Table 2: Teeth3DS dataset statistics

3.1.2. FDI World Dental Federation Notation System

The Fédération Dentaire Internationale (FDI) two-digit notation system [9] [10] represents the international standard for identifying individual teeth in dental records, datasets and software systems. The notation allows us to further explore the objectives of the MICCAI 2022 3D Teeth Scan Segmentation and Labelling Challenge, which is an essential part of the proposed pipeline that is being implemented. Understanding the standard notation is needed for us to interpret per-tooth segmentation labels correctly, which is often varied [11] due to distinct positions and shape variation across patients, abnormalities like crowding and holes, damaged teeth and the presence of non-teeth objects.

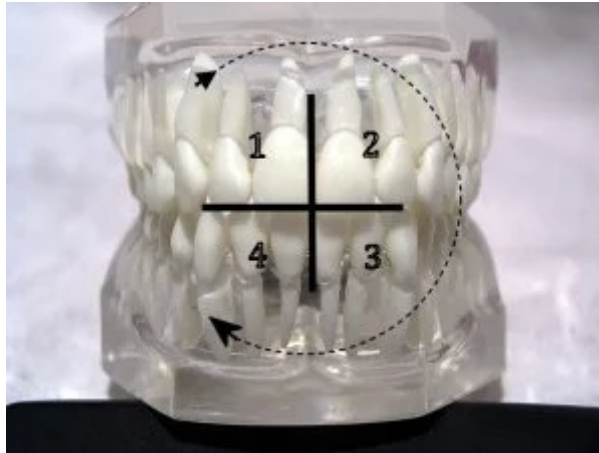


Figure 1: Dental quadrant with respect to dental midline [8]

The system is meant to identify and then divide the jaw scans into four quadrants with respect to the dental midline, which is a vertical line between the upper and lower central incisors and will act as the reference point. The four quadrants:

- Quadrant 1: Upper right (left of midline in anterior view from patient's perspective)
- Quadrant 2: Upper left (right of midline in anterior view from patient's perspective)
- Quadrant 3: Lower left (right of midline in anterior view from patient's perspective)
- Quadrant 4: Lower right (left of midline in anterior view from patient's perspective)

Within each quadrant, the teeth are segmented first by numbering 1-8 from the midline outward to the back of the jaw. The sequence of numbers with their associated teeth naming convention is as follows: 1: central incisor, 2: lateral incisor, 3: canine, 4: first premolar, 5: second premolar, 6: first molar, 7: second molar, 8: third molar or most well known as the wisdom tooth. The FDI label is then formed by concatenating the quadrant number with the associated position. For example, the upper right central incisor would be identified as FDI 11, the upper left canine would be identified as 23 and so on.

Position	Quadrant 1 (Upper Right)	Quadrant 2 (Upper Left)	Quadrant 3 (Lower Left)	Quadrant 4 (Lower Right)
1 (Central Incisor)	11	21	31	41
2 (Lateral Incisor)	12	22	32	42
3 (Canine)	13	23	33	43
4 (First Premolar)	14	24	34	44
5 (Second Premolar)	15	25	35	45
6 (First Molar)	16	26	36	46
7 (Second Molar)	17	27	37	47
8 (Third Molar)	18	28	38	48

Table 3: FDI Notation Quick Reference Table

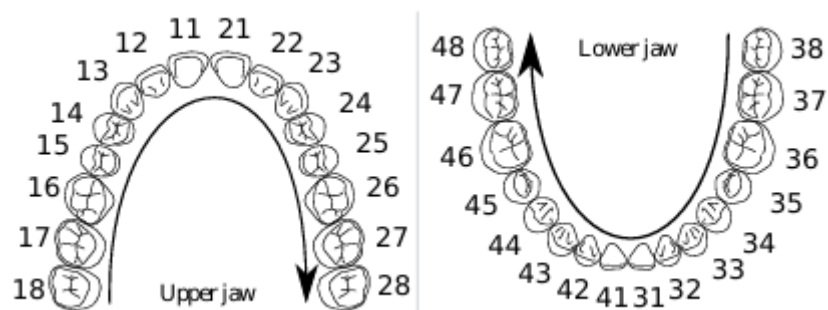


Figure 2: FDI Notation Quick Reference Diagram [11]

3.1.3 Mesh File Format (.obj)

Each scan is stored as a Wavefront OBJ file - a plain text format that is supported by 3D software. The Teeth3DS OBJ files contain two types of entries: vertex definitions and face definitions.

```
v -22.25491000 16.01346432 -91.67618546 0.50196078 0.50196078 0.50196078
v -19.28616670 1.25787788 -87.94231619 0.50196078 0.50196078 0.50196078
v -6.88169350 -3.94281962 -92.77158075 0.50196078 0.50196078 0.50196078
v -16.79466472 6.34788467 -88.32549515 0.50196078 0.50196078 0.50196078
v -1.14610056 -20.33714221 -91.10306800 0.50196078 0.50196078 0.50196078
f 611 293 1037
f 200 1051 609
f 1141 989 247
f 418 224 1188
f 1246 655 812
```

Figure 3: Snippet of mesh file format from Teeth3DS (.obj)

Each vertex line encodes seven values. The vertex line is first defined by the character “v” at the start of its line, followed by three spatial coordinate values and another three colour values, which gives the form:

v	x	y	z	r	g	b
v	-19.28616670	1.25787788	-87.94231619	0.50196078	0.50196078	0.50196078

The first three values are the vertex position in millimetres, in the coordinate frame of the intraoral scanner. The last three values are the per-vertex RGB colour encoded as the floating-point values in the range of 0 to 1.

Each face line defines one triangular polygon by referencing three vertex indices. Faces connect three vertices to form a triangle on the mesh surface.

f	i	j	k
f	1141	989	247

3.1.4 Segmentation Annotation Format (.json)

Each mesh file is paired with a JSON annotation file that stores the per-vertex tooth labels. The structure is as follows:

```
{  
  "id_patient": "01346914",  
  "jaw": "upper",  
  "labels": [0, 0, 16, 16, 25, 0, 11, 14, ...],  
  "instances": [0, 0, 3, 3, 8, 0, 1, 2, ...]  
}
```

Field	Type	Description
id_patient	string	Unique patient identifier, matching the folder name
jaw	string	"upper" or "lower" jaw.
labels	integer array	Per-vertex FDI code. The length of the array is equivalent to the number of vertices in the .obj file. 0 = gingiva
instances	integer array	Per-vertex instance ID. It groups vertices belonging to the same tooth region.

Table 4: Segmentation Annotation json file field description

The i -th vertex of the .obj file corresponds to the value present in labels[i], which gives the FDI teeth code that the particular vertex belongs to. A value of 0 indicates the particular

vertex lies on the gingiva. Upper jaw scans contain labels drawn from the set $\{0, 11-17, 21-27\}$ and lower jaw scans from $\{0, 31-37, 41-47\}$, depending on which teeth are present. Missing teeth result in the complete absence of their FDI code from the label array.

3.2 3DTeethLand Landmark Dataset

The 3DTeethLand dataset [13] provides the anatomical landmark annotations for a subset of the Teeth3DS scans. The dataset was released as part of the MICCAI 2024 3D TeethLand: 3D Teeth Landmarks Detection Challenge. Each annotated scan is accompanied by a keypoint JSON file (`{stem}_kpt.json`) containing a list of landmark entries that corresponds with an (x, y, z) coordinate in mm, a class label from the six landmark types and a tooth FDI identifies. Ground truth annotations were produced by professional annotators with the help of Udini, with annotators identifying and labelling the tooth landmark class on each tooth. To ensure accuracy and reliability of the landmark annotations, the annotations were also reviewed and verified by three clinical evaluators of the challenge.

Annotated teeth landmarks present in the dataset include:

Mesial & Distal points [13, 17]: The mesial surface of a tooth refers to the proximal surface that faces towards the midline of the dental arch, while the distal surface is its counterpart, which is the point that is oriented away from the midline. The two landmark points are annotated on their respective surfaces and together define the primary axis where the tooth is located within the arch. In the 3D TeethLand dataset, exactly one mesial and one distal point is annotated per tooth and is the most consistently defined landmark across all tooth types.

Cusp points [13, 17]: A cusp is the pointed, rounded peak present on the occlusal chewing surface of a specific tooth. This landmark is usually placed at the apex of each cusp and is critical for characterising the occlusal relationship between opposing teeth in the upper and lower jaw arches during biting and chewing. Unlike other landmarks, the number of cusp points is expected to fluctuate depending on tooth morphology. This variability makes cusp points detection the most challenging landmark class in the dataset.

Inner & Outer Points [13, 17]: the inner and outer landmark points are defined as the cervical boundary of the tooth, which is designated as the point and region where the tooth crown meets the gingival tissue. The inner point is oriented towards the tongue or palate, while the outer point faces the cheeks and lips. The two points when taken together establish a

transverse reference axis across the cervical region of the tooth, giving an informative estimation of the three-dimensional orientation and geometry of the tooth within the arch. Exactly one inner and one outer point are expected to be annotated per tooth, regardless of their morphology.

Facial Axis Point [13, 17]: this landmark is placed at the geometric centre of the facial surface of each tooth, which will be the surface that is directly visible from the front of the mouth. In the context of this dataset, the facial axis point provides a stable and morphology-independent reference that complements the orientation and geometric information captured by the inner and outer points. One facial axis point is annotated per tooth across all the tooth types.

In summary, the total number of landmarks annotated per tooth will differ according to tooth morphology, which can be heavily influenced by the variable number of cusp points of certain teeth. The other five non-cusp landmarks are expected to be present once on each tooth, which gives a baseline of expecting at least 5 landmarks present for each tooth. Cusp annotations are expected to add to this baseline, with incisors contributing no additional points; canines have one, premolars have two, and molars have around three to five cusp points, depending on the specific tooth. This structure ensures that the total landmark count per tooth can be expected to range from five on incisors in front of the jaw to nine or more on molars at the rear of the jaw.

Landmark Class	Expected Count per tooth
Mesial	1
Distal	1
Cusp	0 on teeth like incisors, 1-2 on premolars, 2-5 on molars
Inner Point	1
Outer Point	1
Facial Point	1

Table 5: Expected count of landmark class per tooth

3.2.1 Landmark Annotation Format (_kpt.json)

Each landmark file follows a JSON schema containing a flat list of annotated landmark objects, each representing a single point:

```
{
  "version": "1.1",
  "description": "landmarks",
  "key": "01ECZRY6_lower.obj",
  "objects": [
    {
      "key": "uuid_0",
      "class": "Mesial",
      "coord": [3.3149259601813394, -15.242925570061622, -83.04390557810555]
    },
    {
      "key": "uuid_1",
      "class": "Mesial",
      "coord": [3.702200849374723, -16.37522935580949, -83.25572423662976]
    },
    {
      "key": "uuid_2",
      "class": "Mesial",
      "coord": [8.755533685215216, -14.401544782844931, -83.63069598569354]
    },
    {
      "key": "uuid_3",
      "class": "Mesial",
      "coord": [13.672313449352266, -11.99716295661562, -85.90744671931076]
    },
    {
      "key": "uuid_4",
      "class": "Mesial",
      "coord": [16.68674271881639, -6.17873438142966, -88.3769716914598]
    }
  ]
}
```

Figure 4: Snippet of content within landmark annotation JSON

```
{
  "version": "1.0",
  "description": "landmarks",
  "key": "01F4JV8X_upper.obj",
  "objects": [
```

```

{
  {
    "key": "uuid_0",
    "class": "Mesial",
    "coord": [2.259, -20.359, -83.598]
  },
  {
    "key": "uuid_14",
    "class": "Distal",
    "coord": [8.267, -19.263, -83.642]
  },
  {
    "key": "uuid_27",
    "class": "Cusp",
    "coord": [18.082, 21.833, -89.929]
  }
]
}

```

Field	Description
key (top-level)	Filename of the associated mesh, linking annotations back to the correct .obj
objects	Array of all landmark points for this scan
objects[i].key	Unique UUID string that identifies the landmark
objects[i].class	Mesial, Distal, Cusp, InnerPoint, OuterPoint, FacialPoint
objects[i].coord	3D coordinates (x, y, z) in millimetres, in the same scanner coordinate frame as the paired .obj

Table 6: Description of fields in the landmark annotation JSON

Critically, the annotation file does not state which FDI tooth each landmark belongs to. All landmark points for a scan are stored as a single flat, unordered list. In addition, some annotation files in the training split include additional metadata fields, which include `labelerLogin`, `createdAt`, `updatedAt`, recording the annotator's identity and timestamp. These fields carry no geometric information and are discarded during data loading.

Chapter 4 Methodology

4.1 System Architecture Overview

The project implements a fully automated two-stage pipeline for 3D dental segmentation and landmark detection from raw intraoral mesh scans. The pipeline's input expects a 3D mesh of either the upper or lower dental jaw in OBJ format and produces precise 3D coordinates of six anatomical landmark classes for every tooth present in the scan, accompanied by colour-coded visualisation files in ply format.

The two stages are functionally and architecturally distinct. Stage 1 of the pipeline addresses tooth instance segmentation, where, given the unsegmented jaw in a point cloud, it assigns each point a standardised FDI tooth label. Meanwhile, Stage 3 addresses landmark localisation, where, given the segmented teeth according to their point clouds from stage 1, the positions of the anatomical landmarks for each individual tooth is detected and labelled correctly.

Moreover, an intermediate stage 2 - the coordinate-space bridge, is needed to connect the two networks to ensure consistent normalisation is passed between the networks. Following Stage 3, the output results are processed by aggregating the predictions into clean and meaningful landmark coordinates.

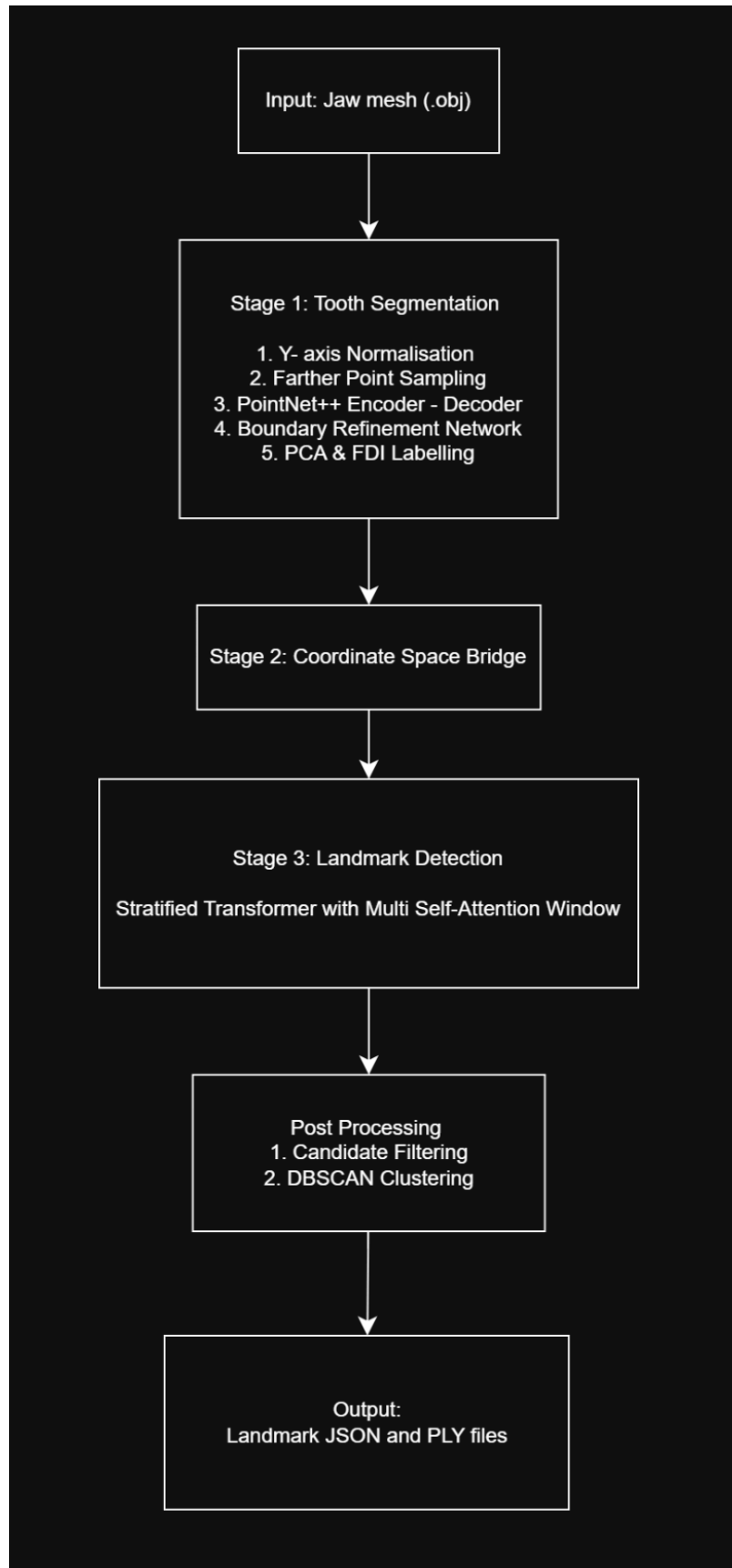


Figure 5: Architecture Flow Diagram

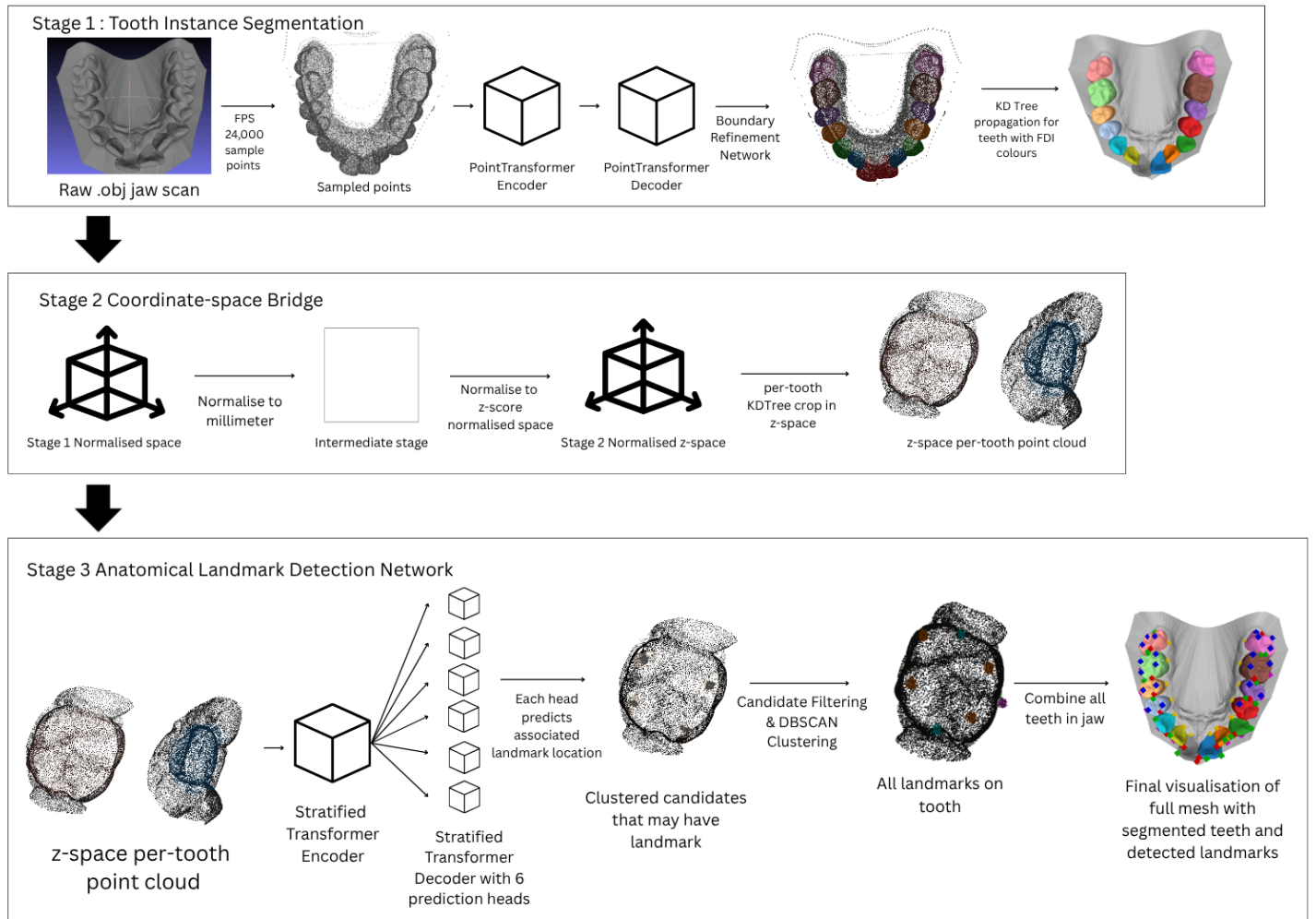


Figure 6: Visualised step-by-step pipeline architecture

4.2 Stage 1: Tooth Instance Segmentation

4.2.1 Input Parsing & Normalisation

The input to the pipeline is a triangular mesh stored in the Wavefront OBJ format, as produced by the Teeth3DS dataset. The mesh is parsed to extract each vertex's x, y, and z coordinates, along with the associated per-vertex surface normals which are computed from the mesh geometry by Open3D. Duplicate vertex removal is performed immediately to eliminate artificial point clusters that may bias subsequent sampling and inflate point counts without adding geometric information.

```
_, mesh = gu.read_txt_obj_ls(stl_path, ret_mesh=True, use_tri_mesh=True)
mesh = mesh.remove_duplicated_vertices()
```

The coordinates are stored as physical millimetres, but their absolute position and geometry will differ across patients, reflecting differences in jaw size, scanner placement and coordinate frame. Hence, a normalisation step is taken to ensure the network is trained to generalise across all instances. The preprocessing step normalises the jaw along the y-axis, which is the axis aligned in the occlusal-gingival direction. The scan is first centred around its mean position, then rescaled. Critically, the same scaling factors are applied equally to the X-axis and Z-axis, which allows the true three-dimensional shape of the scan to be preserved without distortion, while standardising the overall size of the mesh scan.

Some patients' scans have fewer than 24000 vertices after deduplication. Since the segmentation network requires exactly 24000 input points, inadequate scans would require upsampling. Rather than replicating existing points, which defeats the purpose of removing the duplicates in the prior step, a midpoint subdivision is applied instead.

```
if np.asarray(mesh.vertices).shape[0] < 24000:
    mesh = mesh.subdivide_midpoint(number_of_iterations=1)
    bdl_feats = np.array(np.concatenate([np.array(mesh.vertices), np.array(mesh.vertex_normals)], axis=1))
else:
    bdl_feats = org_feats.copy()
```

This step inserts a new vertex at the midpoint of every edge in the mesh, producing a finer tessellation of the same surface. This step approximately quadruples the vertex count, ensuring all scans exceed the 24000 point requirement needed for the next step.

4.2.2. Farther Point Sampling (FPS)

```
def fps(xyz, npoint):
    if xyz.shape[0] < npoint:
        raise "new fps error"
    xyz = torch.from_numpy(np.array(xyz)).type(torch.float).cuda()
    idx = pointops.furthestpointselect(xyz, torch.tensor([xyz.shape[0]]).cuda().type(torch.int), torch.tensor([npoint]).cuda().type(torch.int))
    return torch_to_numpy(idx).reshape(-1)
```

```
class FurthestSampling(Function):
    @staticmethod
    def forward(ctx, xyz, offset, new_offset):
        """
        input: xyz: (n, 3), offset: (b), new_offset: (b)
        output: idx: (m)
        """
        assert xyz.is_contiguous()
        n, b, n_max = xyz.shape[0], offset.shape[0], offset[0]
        for i in range(1, b):
            n_max = max(offset[i] - offset[i-1], n_max)
        idx = torch.cuda.IntTensor(new_offset[b-1].item()).zero_()
        tmp = torch.cuda.FloatTensor(n).fill_(1e10)
        pointops_cuda.furthestpointselect_cuda(b, n_max, xyz, offset, new_offset, tmp, idx)
        del tmp
        return idx

furthestpointselect = FurthestSampling.apply
```

After ensuring all the meshes have sufficient density, they are resampled to a fixed cardinality of 24,000 points using Farthest Point Sampling to ensure consistent input size across all scans. Uniform random subsampling would produce uneven spatial coverage, which would overfit according to denser regions and underfit the sparse regions. The methodology of Farthest Point Sampling is as follows:

1. Select a random seed point as the initial set S .
2. Compute the distance from every remaining point to its nearest point in S .
3. Select the point with the greatest distance to S .
4. Repeat until $|S| = 24000$, i.e. set S to have 24000 points.

Consequently, set S will contain the points that will form a spatially uniform subsampling of every region of the jaw surface, regardless of how dense or sparse different regions of the original mesh are. For context, random subsampling would have underrepresented sparse but geometrically important areas such as cusp tips and interproximal contact points.

4.2.3 Point Transformer Encoder-Decoder Segmentation Network

The 24000 point cloud is then passed to the hierarchical encoder-decoder segmentation network. This architecture is adopted from TGNNet, developed by the CGIP Team [12], the network predicts the per-point instance label associated with each vertex. The network adopts a U-Net structure and follows a hierarchical architecture, with the encoder progressively compressing the point cloud into a compact global representation and the decoder expands it back to the original resolution, predicting a label at each point. The encoder consists of fiveset abstraction layers, each of which performs:

1. **Sampling:** Select a set of centroid points from the input using FPS (at a lower density than the input).
2. **Grouping:** For each centroid, find all input points within a fixed radius ball and form a local neighbourhood set.
3. **Learning:** Apply a Point Transformer self-attention over the relative coordinates and features of each point within the local neighbourhood, then each point computes attention weights by comparing its query against the keys of its neighbours and aggregates their values with a softmax-weighted sum.

A shared multi-layer perceptron processes patches using the same set of learned weights to generalise across spatial positions. A max-pooling step is applied and extracts the most important feature from each local neighbourhood.

Layer	Stride	Input Points	Output Points	Neighbours (nsample)	Transformer Blocks	Feature Dim
SA-1 (enc1)	1	24,000	24,000	36	2	32
SA-2 (enc2)	4	24,000	6,000	24	3	64
SA-3 (enc3)	4	6,000	1,500	24	4	128
SA-4 (enc4)	4	1,500	375	24	6	256
SA-5 (enc5)	4	375	~93	24	3	512

Table 7: Point Transformer Encoder Layer Specifications

In the first SA layer with a stride value of 1, there is no point reduction and the points are projected to 32 dimensions without subsampling. Subsequent layers select 1 in every 4 points via subsampling, reducing the count of points by 4 times. By the end of the 4th layer, the roughly 94 remaining points each hold a feature vector that encodes both local surface detail and global jaw context. This is reflected by the feature dim that doubles at each stage. As the point count shrinks, each remaining point will carry richer information to compensate.

Layer	Input Points	Output Points	Feature Dim	Skip Connection From
FP-5 (dec5)	~93	~93	512	— (bottleneck self-process)
FP-4 (dec4)	~93	375	256	SA-4 (enc4)
FP-3 (dec3)	375	1,500	128	SA-3 (enc3)
FP-2 (dec2)	1,500	6,000	64	SA-2 (enc2)
FP-1 (dec1)	6,000	24,000	32	SA-1 (enc1)

Table 8: Point Transformer Decoder Layer Specifications

The decoder then uses four feature propagation (FP) layers that restore the compressed version to the original point cloud. This is done by interpolating features from coarser to denser point sets from the matching encoder levels at each layer. As the encoder compresses the points, it deliberately throws away much finer details for the leverage to learn a more global context. Hence, skipped connections are adopted within the decoders to save the local details again for more precise boundaries drawn. The skip connection works by simply saving a copy of the intermediate output at each encoder level, which gets plugged back in at the matching decoder level. Hence, at each FP layer, the global context defined from the decoder and the finer feature from the skip connections are concatenated, passed through an MLP, to restore the refined features for that resolution level.

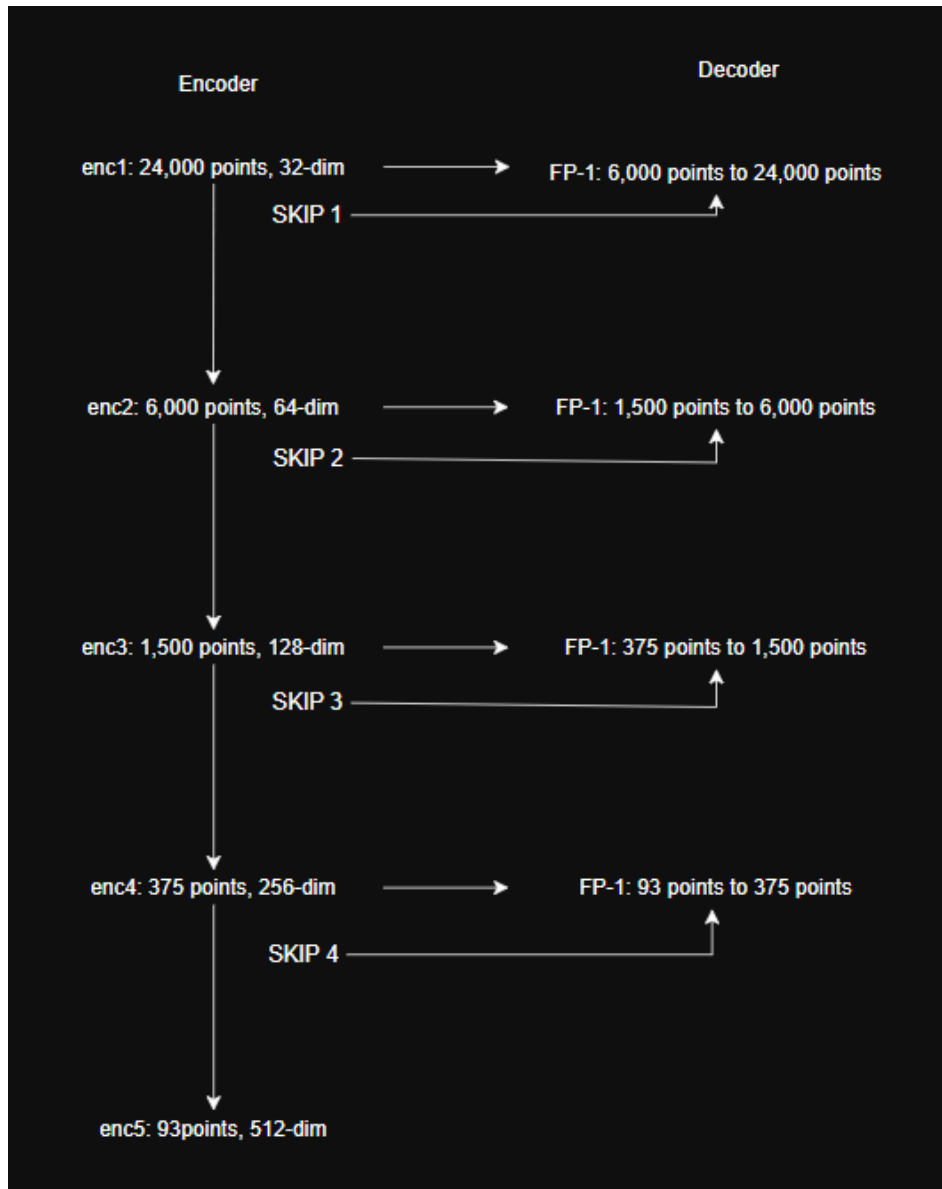


Figure 7: Point Transformer Encoder - Decoder Visualisation

```

first_xyz = first_results["ins"]["full_ins_labeled_points"][:, :3]
first_ps_label = first_results["ins"]["full_ins_labeled_points"][:, 3].astype(int)
first_sem_xyz = first_results["sem_1"]["full_labeled_points"][:, :3]
first_sem_label = first_results["sem_1"]["full_labeled_points"][:, 3]

```

Ultimately, the network outputs two parallel label arrays for all 24,000 sampled points. The `first_ps_label` variable, which constitutes the instance label that groups the points into a region, clarifies the particular tooth instance the point belongs to. Meanwhile, the `first_sem_label` variable classifies the type of tooth with a numbered label, indicating the type of tooth the point belongs to.

4.2.4 Boundary Aware Deep Learning Network

The boundary-aware refinement network is a targeted post-correction module designed to sharpen inter-tooth boundary assignments. The first network is generally accurate when it comes to the interior of each tooth, but may make mistakes at the edges. Hence, this proposed network is a smaller Point Transformer network and will focus solely on points near tooth boundaries to clearly define the boundaries of teeth, which may look similar and shared from a local neighbourhood perspective. This network utilises the first Point Transformer Segmentation network's prediction as additional inputs to correct and clarify edge uncertainties.

```
sampled_boundary_feats, sampled_boundary_seg_label, only_boundary_feats, only_boundary_seg_label = self.get_boundary_sampled_feats(
    first_results["ins"]["full_ins_labeled_points"][:,3],
    bd1_feats,
    sampled_feats,
    None
)

def get_boundary_sampled_feats(self, point_labels, org_feats, sampled_feats, sample_output_features):
    xyz_cpu = sampled_feats[:,3].copy() # N,3
    tree = KDTree(xyz_cpu, leaf_size=40)

    bd_labels = np.zeros(org_feats.shape[0]) # N
    ps_labels = np.zeros(org_feats.shape[0]) # N
    near_points = tree.query(org_feats[:,3], k=40, return_distance=False, )

    labels_arr = point_labels[near_points]
    label_counts = gu.count_unique_by_row(labels_arr)
    label_ratio = label_counts[:, 0] / 40.

    bd_labels[label_ratio < self.config["boundary_sampling_info"]["bd1_ratio"]] = 1

    k_1_near_points = tree.query(org_feats[:,3], k=1, return_distance=False)
    ps_labels = point_labels[k_1_near_points[:,0]].reshape(-1,1)

    bd_org_feat_cpu = org_feats[bd_labels==1, :]
    bd_org_ps_label_cpu = ps_labels[bd_labels==1, :]
    bd_org_feat_cpu, bd_org_ps_label_cpu = gu.resample_pcd([bd_org_feat_cpu, bd_org_ps_label_cpu], self.config["boundary_sampling_info"]["num_of_bd1_points"], "uniformly")
    non_bd_org_feat_cpu = org_feats[bd_labels!=1, :]
    non_bd_org_ps_label_cpu = ps_labels[bd_labels!=1, :]
    non_bd_org_feat_cpu, non_bd_org_ps_label_cpu = gu.resample_pcd([non_bd_org_feat_cpu, non_bd_org_ps_label_cpu],
        self.config["boundary_sampling_info"]["num_of_all_points"]-bd_org_feat_cpu.shape[0], "fps")

    results_feat_cpu = np.concatenate([bd_org_feat_cpu, non_bd_org_feat_cpu], axis=0)
    results_label_cpu = np.concatenate([bd_org_ps_label_cpu, non_bd_org_ps_label_cpu], axis=0)

    if sample_output_features is not None:
        sample_output_features = gu.torch_to_numpy(sample_output_features)[0,:].T

        near_points = tree.query(results_feat_cpu[:,3], k=3, return_distance=True)
        near_points_idxes = near_points[1]
        near_points_prop = near_points[0]
        near_points_prop = near_points_prop / np.sum(near_points_prop,axis=1).reshape(-1,1)

        nn_features = sample_output_features[near_points_idxes] * near_points_prop.reshape(24000,3,1)
        nn_features = nn_features.sum(axis=1)
        nn_features = nn_features.astype('float32')
        results_feat_cpu = np.concatenate([results_feat_cpu, nn_features], axis=1)

    return results_feat_cpu, results_label_cpu, bd_org_feat_cpu, bd_org_ps_label_cpu
```

First, with reference to the codebase pasted above, the network selects only the points that sit near the boundary between two different instance labels. The network processes the boundary points at constant resolutions without compressing the data, and takes in the first Point Transformer network's instance label predictions as additional feature channels. It learns to correct systematic errors made at boundaries and improve the precision.

Layer	Stride	Input Points	Output Points	Neighbours	Transformer Blocks	Feature Dim
SA-1 (enc1)	1	N_boundary	N_boundary	36	2	16
SA-2 (enc2)	1	N_boundary	N_boundary	24	3	32

Table 9: Boundary Refinement Encoder Specifications

At each SA layer, the encoder finds the nearest sample points around it to learn which of the neighbours are most relevant and update its own feature vector. The encoder here is meant to enrich each point’s feature vector by gathering context from the local neighbourhood. After SA-2, every boundary point knows not just its own position and normal, but also the surrounding geometry

Decoder

Layer	Input Points	Output Points	Feature Dim
FP-2 (dec2)	N_boundary	N_boundary	32
FP-1 (dec1)	N_boundary	N_boundary	16

Table 10: Boundary Refinement Decoder Specifications

Subsequently, the decoder then merges the features from both encoder layers, combining the 16-dimensional local context of SA-1 with the broader 32-dimensional local context from SA-2 to produce more refined features along with the previous input. Finally, the precise instance and semantic labels of the boundary points are overwritten with the refined BDL labels.


```

final_ins_points = np.concatenate([first_xyz, bdl_xyz], axis=0)
mod_bdl_ps_label = mod_bdl_ps_label.astype(int)
final_ins_labels = np.concatenate([first_ps_label.reshape(-1,1), mod_bdl_ps_label.reshape(-1,1)], axis=0)
final_ins_labels = final_ins_labels.astype(int)
final_sem_labels = np.concatenate([new_sem_labels.reshape(-1,1), mod_bdl_sem_label.reshape(-1,1)], axis=0)
final_sem_labels = final_sem_labels.astype(int)

```

Finally, an upsampling of point to 24,000 sample points of the original mesh vertices count, to ensure all vertices from the original mesh have a label. Therefore, a K-Dimensional Tree (KDTree) is implemented to find the single nearest sampled point for each of the original vertices and allow the vertices to adopt the corresponding semantic and instance labels of the sampled point. Subsequently, the transfer with per-vertex labelling of all points in the original mesh is completed.

```

tree = KDTree(final_ins_points, leaf_size=2)
near_points = tree.query(org_feats[:, :3], k=1, return_distance=False)
result_ins_labels = final_ins_labels.reshape(-1)[near_points.reshape(-1)].reshape(-1,1)
result_sem_labels = final_sem_labels.reshape(-1)[near_points.reshape(-1)].reshape(-1,1)

```

4.2.5 FDI Label Assignment

After the two networks, the relevant FDI code of each segmented tooth with the information from each vertex instance number and the semantic tooth type is computed. The final result of the stage 1 pipeline is a complete per-vertex FDI label map, with every tooth correctly identified by its corresponding two-digit code.

4.3 Stage 2: Coordinate Space Bridge

4.3.1 Motivation

The segmentation network from stage 1 and the landmark detection network of stage 3 were trained under different coordinate conventions respectively. The landmark detection network is trained on a standardised z-score normalised space with the 3D Teethland dataset, whereas the segmentation network is trained with a normalised Y-axis.

Neither coordinate system of the two networks in the pipeline is convertible to the other by a simple scalar multiplication, as Stage 1 is normalised to a fixed range of 1.8, while Stage 3 is normalised to a fixed statistical dispersion. Hence, the coordinate bridge performs a two-step data pre-processing transformation before moving the mesh data for training in the landmark detection segment of the pipeline:

Stage 1's normalised space $\rightarrow$ physical mm coordinate space in Stage 2 $\rightarrow$ Stage 3's z-score space.

4.3.2 Scale Factor Derivation & Centroid Conversion

```
def get_scale_factor(original_vertices_mm: np.ndarray) -> float:
    """
    Derive how many millimetres correspond to one Stage 1 normalised unit.

    Args:
        original_vertices_mm: (N, 3) float array of original mesh vertices in mm.

    Returns:
        scale_factor: float - mm per Stage 1 unit.
    """
    y_extent_mm = float(
        original_vertices_mm[:, 1].max() - original_vertices_mm[:, 1].min()
    )
    if y_extent_mm < 1e-6:
        raise ValueError("Original mesh has zero Y extent - check input mesh.")
    return y_extent_mm / TGNET_Y_RANGE
```

Scale factor derivation: data_bridge.py — lines 35–40

With that, the normalised stage 1 results are reversed by converting the mesh scan units back to millimetres. The scale factor used above would quantify how many physical millimetres would correspond to one unit in the normalised coordinate system of stage 1. As the entire

jaw along the y-axis was normalised to exactly 1.8 units in stage 1, dividing the original extent by 1.8 recovers the equivalent millimetre scale to one normalised unit of the segmentation network, giving a proper mm-per-unit ratio. Then, the centroid of each tooth in the normalised space of stage 1 is converted to the z-score space of stage 2 as follows.

```
tooth_norm_xyz = sampled_xyz[tooth_mask] # TNet space
centroid_tgnet = tooth_norm_xyz.mean(axis=0) # (3,)
centroid_norm = ((centroid_tgnet + 0.8) * scale_factor + min_y_c) / Z_SCORE_STD
```

Centroid Conversion: pipeline/combined_pipeline.py - lines 197-199

The formula of $(\text{centroid} + 0.8) \times \text{scale_factor} + \text{min_y_c}$ inverts the segmentation network's normalisation, recovering the centroid in millimetre-based coordinates.

The three operations decompose as follows:

- $+ 0.8$ — un-shifts the TNet range from $[-0.8, 1.0]$ back to $[0, 1.8]$
- $\times \text{scale_factor}$ — converts back to millimetres relative to the centred origin
- $+ \text{min_y_c}$ — restores the centering offset
- $/ \text{Z_SCORE_STD}$ — applies 3DTeethLand's z-score scaling

The scalar `min_y_c` accounts for the fact that Stage 1's normalisation procedure is first to subtract the 3D mean, then normalise by the Y-range. The subtraction of the 3D mean displaces the Y-minimum from its raw value, and `min_y_c` captures exactly this displacement for the Y-axis.

4.3.3 Full Jaw Z-Score Normalisation

In parallel with the centroid conversion, the complete set of original mesh vertices is normalised into z-score space to serve as the point pool from which tooth crops are drawn.

```
jaw_mean = orig_xyz_mm.mean(axis=0) #
jaw_norm = (orig_xyz_mm - jaw_mean) / Z_SCORE_STD
```

Full jaw z-score normalisation: pipeline/combined_pipeline.py - line 161-162

The full jaw point cloud is now successfully transformed and normalised according to the z-score space needed for stage 3 of the pipeline.

4.3.4 Per Tooth Crop Extraction with KDTree

Moving on, the landmark detection pipeline is expected to operate on individual tooth crops rather than processing the entire jaw simultaneously. This architectural design is motivated by a single factor where landmark positions are locally defined relative to a single tooth's anatomy, and do not require the global jaw context, as reflected in Chapter 3.2 3D Teethland Landmark Dataset. This also significantly reduces the input size to the network, which may inflate the complexity.

```
def __init__(
    self,
    tgnet_fps_ckpt: str,
    tgnet_bdl_ckpt: str,
    landmark_ckpt: str,
    device: str = "cuda",
    crop_k: int = 12000,
):
    self.device = device
    self.crop_k = crop_k

# Build KDTree in z-score normalised space for tooth-neighbourhood lookup
from sklearn.neighbors import KDTree as _KDTree
norm_tree = _KDTree(jaw_norm, leaf_size=16)

crop_idx = norm_tree.query(
    centroid_norm[None], k=self.crop_k, return_distance=False
)[0] # (crop_k,)
```

For each labelled tooth, the next process involves finding the top k nearest vertices from its centroid using a KD Tree. This step basically does a k -nearest-neighbour clustering just in a multi-dimensional setting. A default fixed `crop_k` value is defined, which determines the k number of points to be clustered. The KNN query for a fixed point count was preferred over a ball query, a similar clustering from Stage 1, as KNN guarantees a consistent point cloud regardless of the local scan density. For sparse regions, a ball query may capture too few points, which may not capture too many details or too many points in a dense region. With a fixed count at `crop_k = 12000`, the KNN query ensures the pipeline is guaranteed a consistent input tensor shape for every tooth and every scan, which is needed when moving to the Stratified Transformer implemented in Stage 3.

4.3.5 PointTensor Construction with Feature Vector

```
class PointTensor:
    """Implements abstraction of torch.Tensor for batched point cloud data."""

    def __init__(
        self,
        coordinates: TensorType['N', 3, torch.float32],
        features: Optional[TensorType['N', '...', Any]]=None,
        batch_counts: Optional[TensorType['B', Any]]=None,
        batch_indices: Optional[TensorType['N', Any]]=None,
    ):

import PointTensor as PT

crop_xyz_norm = jaw_norm[crop_idxs]          # (crop_k, 3) z-score normalised
crop_normals = orig_normals[crop_idxs]        # (crop_k, 3)

# Centroid offsets in z-score space (matching CentroidOffsetsAsFeatures in training)
centroid_offsets = crop_xyz_norm - centroid_norm # (crop_k, 3)

# Build PointTensor: features = [xyz_norm, normals, centroid_offsets]
crop_xyz_t = torch.from_numpy(crop_xyz_norm).float().to(self.device)
crop_norm_t = torch.from_numpy(crop_normals).float().to(self.device)
cent_off_t = torch.from_numpy(centroid_offsets).float().to(self.device)
features = torch.cat([crop_xyz_t, crop_norm_t, cent_off_t], dim=1) # (N, 9)

pt = PT(
    coordinates=crop_xyz_t,
    features=features,
    batch_counts=torch.tensor([crop_xyz_t.shape[0]], device=self.device),
)
```

Each extracted tooth crop is encoded as a PointTensor, which is a specialised batched point cloud data structure that will match the primary input representation of the Landmark Detection Network. Unlike a plain torch Tensor structure, a PointTensor carries both geometric coordinates, C (shape $(N, 3)$) and per-point feature vectors, F (shape $(N, 9)$) as separate fields, alongside `batch_counts` and `batch_indices` tensors that allow variable-sized point clouds to be processed in a single batched forward pass. This separation between coordinates and features is architecturally important: the coordinates are used exclusively for spatial operations, like KDTREE neighbour queries, FPS downsampling, and CRPE attention position encoding. Meanwhile, the encoded vector features are what the transformer layers actually transform and learn from.

Each point in the crop is assigned a 9-channel feature vector, assembled from three semantically distinct groups

Channels	Content	Design Rationale
0–2	Z-score normalised XYZ	Absolute position of the point within the crop.
3–5	Surface normals	Encodes local surface orientation at each point.
6–8	Centroid offsets (Δx , Δy , Δz)	Signed displacement of each point from the tooth’s centre in z-score normalised space.

Table 11: PointTensor 9-channel feature vector contents & specifications

4.4 Stage 3: Landmark Detection

4.4.1 Overview: Stratified Transformer with Six Output Heads

```
self.landmark_model = LandmarkNet(  
    # training hparams (not used at inference, but required by __init__)  
    lr=0.0006,  
    weight_decay=0.0001,  
    epochs=500,  
    warmup_epochs=5,  
    dbscan_cfg={  
        'max_neighbor_dist': 0.03,  
        'min_points': 40,  
        'weighted_cluster': True,  
        'weighted_average': True,  
    },  
    # StratifiedTransformer architecture (must match checkpoint)  
    in_channels=9,  
    channels_list=[48, 96, 192, 256],  
    out_channels=[1, 4, 4, 4, 4, 4],  
    depths=[3, 9, 3],  
    heads_list=[6, 12, 24],  
    window_sizes=[0.1, 0.2, 0.4],  
    point_embedding={  
        'use': True,  
        'kpconv_point_influence': 0.02,  
        'kpconv_ball_radius': 0.05,  
    },  
    stratified_union=False,  
    downsample_ratio=0.26,  
    max_drop_path_prob=0.3,  
    stratified_downsample_ratio=0.26,  
    crpe_bins=80,  
    transformer_lr_ratio=0.1,  
)  
self.landmark_model.load_state_dict(_ckpt['state_dict'])  
self.landmark_model.eval().to(device)
```

The main architecture behind the anatomical landmark detection deep learning network is a Stratified Transformer, which is a hierarchical, window-based 3D self-attention network which will be adapted for landmark regression. From the previous part, the network will receive the 9-channel PointTensor for each tooth and produce six simultaneous output PointTensor objects for each landmark class.

The Stratified Transformer applies a multi-level hierarchy with scale-specific attention windows, which allows the network to simultaneously reason at fine, intermediate and coarse geometric scales during a single forward pass, allowing the network to understand finer details like surface curvature that are important for cusp tip detections and broader tooth-shaped context for other tasks.

4.4.2 KPConv Point Embedding

Before the transformer does any processing, the raw 9-channel input features are projected into a higher 48- dimensional representation using Kernel Point Convolution (KPConv) embedding block:

```
self.point_embedding = self.init_point_embedding(
    in_channels=channels_list[0],
    out_channels=channels_list[1],
    **{k: v for k, v in point_embedding.items() if k != 'use'},
) if point_embedding['use'] else nn.Identity()
```

```
def init_point_embedding(
    self,
    in_channels: int,
    out_channels: int,
    kpconv_point_influence: float,
    kpconv_ball_radius: float,
) -> nn.Sequential:
    return nn.Sequential(
        KPConv(
            in_channels=in_channels,
            out_channels=out_channels,
            point_influence=kpconv_point_influence,
            ball_radius=kpconv_ball_radius,
        ),
        KPConvResidualBlock(
            in_channels=out_channels,
            out_channels=out_channels,
            point_influence=kpconv_point_influence,
            ball_radius=kpconv_ball_radius,
        ),
    )
```

KPConv aggregates features from each point's spatial neighbourhood within a ball radius of 0.05 in z-score normalised space, or within roughly 0.87 mm in physical space. The aggregated features are stored as a set of learnable kernel points in 3D space. The result is

each point's 9 raw channels are now replaced with a 48-dimensional descriptor which captures its geometrical information of the local neighbourhood. This pre-processing step is essential as the Stratified Transformer instead processes geometrically meaningful representation rather than raw coordinates to develop the network, substantially reducing the number of attention iterations during training of the transformer network.

4.4.2 Stratified Transformer Encoder with Windowed Attention

Following the KPConv transformation, the encoder processes the 48-dimensional point cloud through three successive downsampling stages.

```
self.downsample_blocks = self.init_downsample_blocks(
    channels_list[point_embedding['use']], downsample_ratio,
)
```

```
def init_downsample_blocks(
    self,
    channels_list: List[int],
    downsample_ratio: float,
) -> nn.ModuleList:
    blocks = nn.ModuleList()
    for i in range(len(channels_list) - 1):
        blocks.append(DownsampleBlock(
            in_channels=channels_list[i],
            out_channels=channels_list[i + 1],
            ratio=downsample_ratio,
        ))

    return blocks
```

```
# apply downsampling blocks and transformers to encode input
x_ups = []
blocks = (
    self.downsample_blocks,
    self.transformers[not self.use_point_embedding:],
)
for downsample_block, transformer in zip(*blocks):
    x_ups.insert(0, x)
    x = downsample_block(x)
    x = transformer(x)
```

Each DownsampleBlock (self.downsample_blocks) reduces the point cloud via Farthest Point Sampling and aggregates features from the 16 most nearest neighbours of each surviving point using grouped max-pooling, and projects to the next channel dimension. Across each

downsample stage, the point cloud is progressively compressed while the feature vector grows. This progressive spatial compression allows deepest encoder layer to operate over a global representation of tooth with a high dimensional feature representation at the end of the three downsampling stages.

Encoder Stage	Number of Point Clouds	Feature Width
After KPConv	12,000	48
After Downsample 1	~3,120	96
After Downsample 2	~811	192
After Downsample 3	~211	256

Table 12: Stratified Transformer Encoder Downsampling Process

At each encoder stage, the remaining number of points are processed using a windowed self attention with Contextual Relative Position Encoding (CRPE). Each encoder stage would use window size of $[0.1, 0.2, 0.4]$ in z-score normalised space, which corresponds to approximately 1.7mm, 3.5mm, and 6.9mm. As no single window size would work for all landmark types, three scales are applied to obtain and reason the neighbourhood with fine, medium and coarse resolution in a single forward pass. Each level of the window size captures different landmark classes or neighbourhood information.

```

transformers = nn.ModuleList()
for i in range(len(channels_list)):
    transformer = nn.Sequential()
    for block_idx in range(depths[i]):
        transformer.append(StratifiedTransformerBlock(
            channels=channels_list[i]      # [96, 192, 256]
            heads=heads_list[i],          # [6, 12, 24]
            window_size=window_sizes[i],  # [0.1, 0.2, 0.4]
            union=stratified_union,
            shifted=block_idx % 2 == 1,
            drop_path_prob=drop_path_probs[current_block].item(),
            downsample_ratio=stratified_downsample_ratio,
            crpe_bins=crpe_bins,
        ))
        current_block += 1

# StratifiedTransformer architecture (must match checkpoint)
in_channels=9,
channels_list=[48, 96, 192, 256],
out_channels=[1, 4, 4, 4, 4, 4],
depths=[3, 9, 3],
heads_list=[6, 12, 24],
window_sizes=[0.1, 0.2, 0.4],
point_embedding={
    'use': True,
    'kpconv_point_influence': 0.02,
    'kpconv_ball_radius': 0.05,
}

```

4.4.3 Stratified Transformer Decoder with Six Independent Prediction Heads

Once the encoder has compressed the tooth crop down to the bottleneck compressed representation, it serves as the starting point for six independent decoder paths (one path for each prediction head) to perform the upsampling. Each decoder independently reconstructs full-resolution predictions per point by traversing the encoder's saved intermediate states in reverse order through a series of UpsampleBlock layers.

```

# apply upsampling blocks with skip connections and heads
outs = []
for upsample_blocks, head in zip(self.upsample_blocks, self.heads):
    x = encoding
    for upsample_block, x_up in zip(upsample_blocks, x_ups):
        x = upsample_block(x, x_up)
    x = head(x)

    outs.append(x)

```

Each UpsampleBlock performs two operations. First, it projects the current coarse-resolution features to match the channel dimension of the finer resolution, then uses distance-weighted interpolation to assign features to the larger point set saved during the corresponding encoder stage, before adding the encoder's features directly as a residual skip connection. This ensures that the decoder's final full-resolution output combines both global context (which landmark type is expected here, based on overall tooth shape) and local precision (the exact surface geometry at each point). After three upsample stages, the features are back to the original resolution of 12000 points, each with 48 features. A final linear head projects the 48-dimensional features to the head's output channel, 1 channel for the segmentation head and 4 channels for each of the five landmark heads.

```
out_channels=[1, 4, 4, 4, 4, 4],
# [seg, MesialDistal, FacialPoint, OuterPoint, InnerPoint, Cusp]
```

The 4-channel output of each landmark head encodes a prediction at every surface point, with channel 0 predicting the distance from that point to its nearest corresponding landmark defined by the head type, and channels 1–3 predicting the 3D offset vector pointing from that point toward the landmark defined by the head type. Subsequently, every one of the 12000 crop points independently votes on the landmark's location according to their distance from the said landmark.

Head	Landmark Classes	Output Channels
seg_head	Binary tooth mask	1
mesial_distal	Mesial (0) and Distal (1)	4
facial	FacialPoint (2)	4
outer	OuterPoint (3)	4
inner	InnerPoint (4)	4
cusps	Cusp (5)	4

Table 13: Prediction Heads and corresponding output channel dimensions

4.5 Post-Processing and Landmark Clustering

4.5.1 Candidate Landmark Extraction

With each of the five landmark heads producing a 4-channel prediction for every point in the 12000-point crop, the output needs to be processed into discrete landmark position through a filtering and voting procedure.

Channel	Content	Description
$F[:, 0]$	Predicted distance	Distance of surface point from the nearest landmark of this head's type, in z-score normalised units
$F[:, 1]$	Predicted offset Δx	X distance to reach the landmark
$F[:, 2]$	Predicted offset Δy	Y distance to reach the landmark
$F[:, 3]$	Predicted offset Δz	Z distance to reach the landmark

Table 14: Landmark position 4-channel prediction encoding

```

def extract_landmarks(
    head_output: PointTensor,
    dist_thresh: float = DIST_THRESH,
) -> Optional[PointTensor]:
    """
    Post-process one landmark head's output into final landmark coordinates.

    Args:
        head_output: PointTensor where
            head_output.C = (N, 3) point coordinates (mm)
            head_output.F = (N, 4):
                col 0: predicted distance to landmark (mm)
                cols 1-3: predicted (dx, dy, dz) offset to la
            dist_thresh: only points with predicted distance < dist_thresh are u

    Returns:
        PointTensor of detected landmarks:
            .C = (K, 3) landmark positions in mm
            .F = (K, 1) confidence scores in [0, 1]
        Returns None if fewer than 5 candidate points survive the threshold.
    """
    distances = head_output.F[:, 0] # (N,)
    offsets = head_output.F[:, 1:] # (N, 3)

    mask = distances < dist_thresh
    if mask.sum() < 5:
        return None

```

First, points will be filtered by their predicted distance to the landmark, where points predicting a distance below 0.12 normalised units, or approximately 2.1mm physically are retained as candidates eligible for further position voting. Meanwhile if fewer than 5 points are available after the filtering, the head is discarded entirely and the landmark is assumed to not exist on the current tooth entirely.

Each candidate point within the threshold is then able to cast a vote for the landmark's absolute position. Candidate points are assigned higher confidence weights if they have lower predicted distances from the predicted landmark location, since proximity to the landmark would imply a more reliable offset prediction.

```

# Landmark position = mesh point + predicted offset
coords = head_output.C[mask] + offsets[mask]
dists = distances[mask].clamp(0.0, dist_thresh)
weights = (dist_thresh - dists) / dist_thresh #dist_thresh = 0.12

```

4.5.2 DBSCAN Landmark Clustering

The candidate points votes form a cloud of position estimates that must be collapsed into one or more discrete landmark positions. Hence, DBSCAN clustering is used to find dense regions of vote agreement. The strength of DBSCAN is leveraged as the number of landmarks per tooth does not have to be specified before clustering as count of landmarks may differ on a tooth-by-tooth basis. The final landmark position is the confidence-weighted centroid of the accepted cluster with a minimum of 20 points surrounding it. The resulting position is reverted back to physical millimetres.

Each landmark is then stored with its 3D coordinate in mm, anatomical class name, detection confidence score, and the FDI tooth number inherited from Stage 1, forming the final structured output of the pipeline.

4.6 Output Visualisation

Upon completion of the landmark detection and post-processing stages, the pipeline produces two output files per processed scan.

Landmark JSON file: A structured JSON file listing, for each FDI tooth code present in the scan, the detected landmarks of each class with their 3D millimetre coordinates and confidence scores.

Visualisation PLY files: One binary PLY (Polygon File Format) mesh file is produced per landmark class. Each PLY file contains:

1. The original jaw mesh with vertices colour-coded by FDI label. Each distinctly identified tooth would be defined by different colours visually.
2. Octahedral markers are placed at each detected landmark position. Each marker is a regular octahedron (6 vertices, 8 triangular faces) coloured according to their corresponding landmark class. Octahedral markers were chosen for more visually prominent markers during visualisation.

The per-class file separation allows toggling between individual landmark classes in ArcGIS, facilitating visual inspection of detection quality on a class-by-class basis.

The pipeline writes seven files per processed scan, with the stem associated with the original .obj mesh file name, e.g, 01F4JV8X_upper that was used as the sample.

File	Contents
{stem}_landmarks.json	Landmark coordinates, classes, scores, FDI labels
{stem}_mesh.ply	FDI-coloured mesh only, no landmarks
{stem}_colored.ply	FDI-coloured mesh and all landmark octahedra
{stem}_Mesial.ply	Mesial octahedra only
{stem}_Distal.ply	Distal octahedra only
{stem}_Cusp.ply	Cusp octahedra only
{stem}_InnerPoint.ply	InnerPoint octahedra only
{stem}_OuterPoint.ply	OuterPoint octahedra only
{stem}_FacialPoint.ply	FacialPoint octahedra only

Table 15: Output files

```

Saved mesh PLY to C:\Users\lotzi\Downloads\dental_landmark_pipeline\data\output\01F4JV8X_upper\01F4JV8X_upper_mesh.ply
Saved colored PLY to C:\Users\lotzi\Downloads\dental_landmark_pipeline\data\output\01F4JV8X_upper\01F4JV8X_upper_colored.ply
Saving per-class landmark layers:
  FacialPoint: 13 markers -> 01F4JV8X_upper_FacialPoint.ply
  OuterPoint: 14 markers -> 01F4JV8X_upper_OuterPoint.ply
  InnerPoint: 14 markers -> 01F4JV8X_upper_InnerPoint.ply
  Cusp: 21 markers -> 01F4JV8X_upper_Cusp.ply
  Mesial: 14 markers -> 01F4JV8X_upper_Mesial.ply
  Distal: 18 markers -> 01F4JV8X_upper_Distal.ply

```

Figure 8: Output / Creation of octahedra .ply file for respective landmark classes

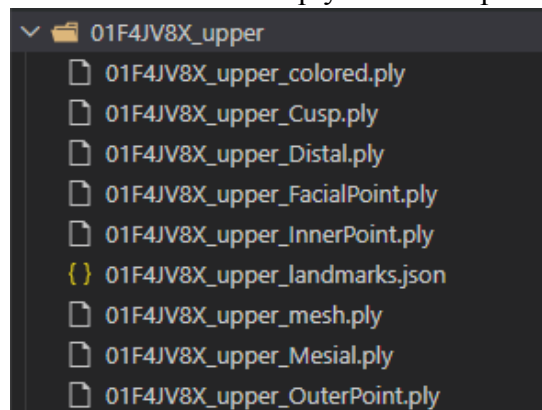


Figure 9: File Directory of sample 01F4JV8X_upper

Chapter 5 Results & Evaluation

5.1 Evaluation Metrics

5.1.1 Tooth Segmentation: Teeth Localisation Accuracy (TLA)

This metric quantifies the precision of tooth placement by measuring the average normalised Euclidean distance between the predicted tooth centres and the actual ground-truth (GT) centres.

- Normalisation: Each distance is adjusted relative to the physical size of the specific ground-truth tooth to maintain consistency across varying tooth dimensions.
- Missing Data Penalty: To penalise failures or omissions, a fixed value of 5 (representing five times the tooth's size) is assigned for any GT tooth that the algorithm fails to identify.
- Aggregation: Because patients have a variable number of teeth, the final TLA score is calculated by averaging these distances across all teeth present in the test datasets.

5.1.2 Tooth Segmentation: Teeth Segmentation Accuracy (TSA)

TSA evaluates the quality of the point cloud boundaries for each tooth.

- Calculation: It is determined by the average F1-score across all individual tooth instances.
- The F1 Formula: The score balances precision (accuracy of the identified tooth points) and recall (completeness of the tooth points) using the harmonic mean:

$$F1 = \frac{2 \times precision \times recall}{precision + recall}$$

5.1.3 Tooth Segmentation: Teeth Identification Rate (TIR)

This metric represents the success rate of correctly recognising and labelling each tooth.

- Criteria for Success: A tooth is considered identified if the predicted centre meets such conditions:
 1. Prediction is located within a distance of less than half the size of the corresponding GT tooth.
 2. Prediction is assigned to the same FDI label as the ground truth.
- TIR is expressed as the total percentage of these successful identifications relative to the entire set of ground-truth.

5.1.4 Tooth Segmentation: Combined Score

Three metrics are combined into a single overall segmentation score:

$$\text{SCORE} = (\text{mean_TLA} + \text{mean_TSA} + \text{mean_TIR}) / 3$$

5.1.5 Landmark Detection: Mean Average Precision (mAP)

The mAP score is calculated by aggregating the per-class Average Precision (AP) score across all 6 landmark categories. Per-class AP is derived from the area under the Precision-Recall curve and averaged across different distance thresholds to give a threshold-averaged AP for each class. mAP would be the mean of the per-class APs across all the 6 landmark classes.

A high mAP requires the pipeline to have achieved high precision and high recall consistently across all landmark types.

5.1.6 Landmark Detection: Mean Average Recall (mAR)

The mAR score is the mean of per-class AR scores, which is calculated by aggregating the Average Recall (AR) for each landmark category. Each AR is determined by the area under the Recall-exp curve.

5.2 Qualitative Results

Running the pipeline on the sample scan 01F4JV8X_upper.obj, produces the following outputs with a single inference pass.

- 14 teeth were segmented correctly and colour-coded properly.

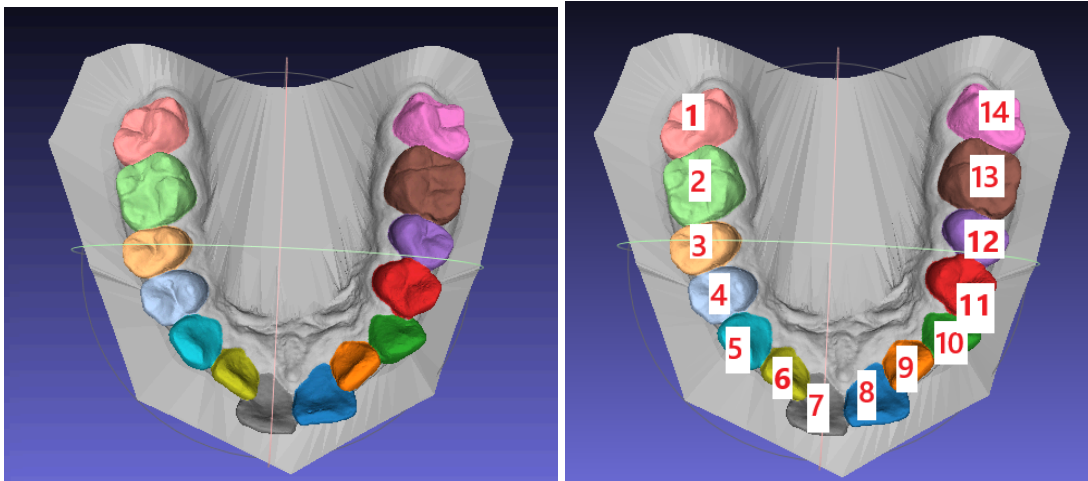


Figure 10: 3D display of 14 segmented teeth with colour coding on 01F4JV8X_upper.obj

- 94 landmarks detected across all 14 teeth.

```
Saved mesh PLY to C:\Users\lotzi\Downloads\dental_landmark_pipeline\data\output\01F4JV8X_upper\01F4JV8X_upper_mesh.ply
Saved colored PLY to C:\Users\lotzi\Downloads\dental_landmark_pipeline\data\output\01F4JV8X_upper\01F4JV8X_upper_colored.ply
Saving per-class landmark layers:
FacialPoint: 13 markers -> 01F4JV8X_upper_FacialPoint.ply
OuterPoint: 14 markers -> 01F4JV8X_upper_OuterPoint.ply
InnerPoint: 14 markers -> 01F4JV8X_upper_InnerPoint.ply
Cusp: 21 markers -> 01F4JV8X_upper_Cusp.ply
Mesial: 14 markers -> 01F4JV8X_upper_Mesial.ply
Distal: 18 markers -> 01F4JV8X_upper_Distal.ply
```

Figure 8: Count of markers per class for 01F4JV8X_upper mesh scan

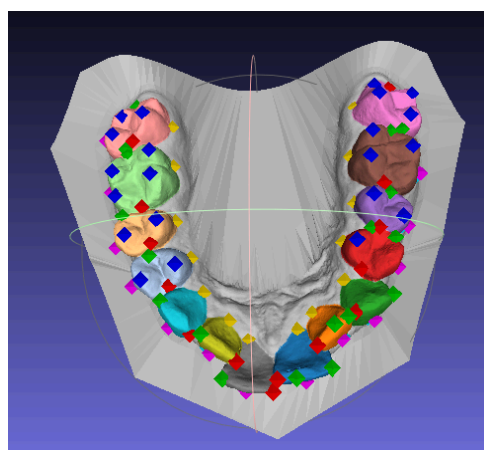


Figure 11: Landmark labelled by colour on 01F4JV8X_upper

5.3 Quantitative Results

```
=====
MICCAI 2022 – Teeth Segmentation (240 scans)
=====
TLA (exp-norm, ↑) : 0.9492 ±0.2178
TSA (micro-F1, ↑) : 0.9838 ±0.0098
TIR (% , ↑)      : 0.4794 ±0.4866
— Challenge Score : 0.8042
=====

MICCAI 2024 – Landmark Detection (240 scans)
=====
mAP (↑) : 0.6220
mAR (↑) : 0.4831

Category          AP          AR
-----
Mesial/Distal     0.6211     0.4966
Cusp              0.6213     0.4965
Inner/Outer       0.6340     0.4880
Facial            0.6005     0.4326

Distal            0.4770
Cusp              0.4965
InnerPoint        0.4759
OuterPoint        0.5001
Distal            0.4770
Cusp              0.4965
InnerPoint        0.4759
OuterPoint        0.5001
FacialPoint       0.4326
=====
```

Figure 12: Evaluation Metrics score in terminal output

Associated Netowrk	Metric	Score
Teeth Segmentation	TLA	0.9492
	TSA	0.9838
	TIR	0.4794
	Comparative Score	0.8042
Landmark Detection	mAP	0.6220
	mAR	0.4831
	AP(cusp)	0.6213
	AR(cusp)	0.4965

Table 16: Evaluation Metric Scores

5.4 Evaluation Scores Discussion

The evaluation results are summarised in Table 16. Firstly, the segmentation network achieves strong performance on TLA (0.9492) and TSA (0.9838) indicating that the predicted tooth centroid are spatially accurate and that the point-level segmentation boundaries closely approximate the ground truth. However, the TIR score of 0.4894 is considerably lower than the other two metrics, reflecting difficulty of the pipeline to correctly assign FDI labels to the teeth point clouds. The assumption is that the network found it hard to learn from underrepresented classes like the wisdom teeth and anomalous dentition cases. Nevertheless, this gap between localisation and segmentation accuracy and identification rate is consistent with the findings from the 3DTeethSeg'22 challenge, where FDI labelling assignment was identified as the most challenging metric across all submitted methods.

Nevertheless, the low TIR score was expected and accounted for in the pipeline, as the pipeline had to proceed from segmentation to anatomical landmark detection. The main premise for the landmark detection network would be for individual teeth in the mesh to be properly segmented from each other. The landmark detection network required the per-tooth crop extraction and did not heavily emphasise on learning from the FDI features from stage 1. Hence, the emphasis to work on this metric was not the highest priority amongst all the evaluation metrics as the combined pipeline required a precise and accurate teeth segmentation contributed from the Stage 1 network.

For landmark detection, the pipeline achieves an mAP of 0.6220 and mAR of 0.4831, compared to the 3DTeethLand challenge's top-performing Radboud method (mAP 0.785, mAR 0.656 [19]). The gap is expected as the 3DTeethLand challenge had solutions that were trained and optimised to focus on specifically landmark detection with the full challenge dataset. The pipeline implemented in this project integrates landmark detection as a downstream component of a combined system, where any errors in Stage 1 segmentation would propagate directly into the tooth crops used for the landmark detection network.

Nevertheless, the score achieved for both networks is satisfactory, considering limitations like GPU-training resources and undergraduate vs postgraduate knowledge gaps. All in all, these results confirm the pipeline's capacity for fully automated end-to-end dental landmark annotation, while identifying clear performance gaps that can be addressed with targeted improvements strategies to the entire pipeline.

5.5 Visualisation Files Output

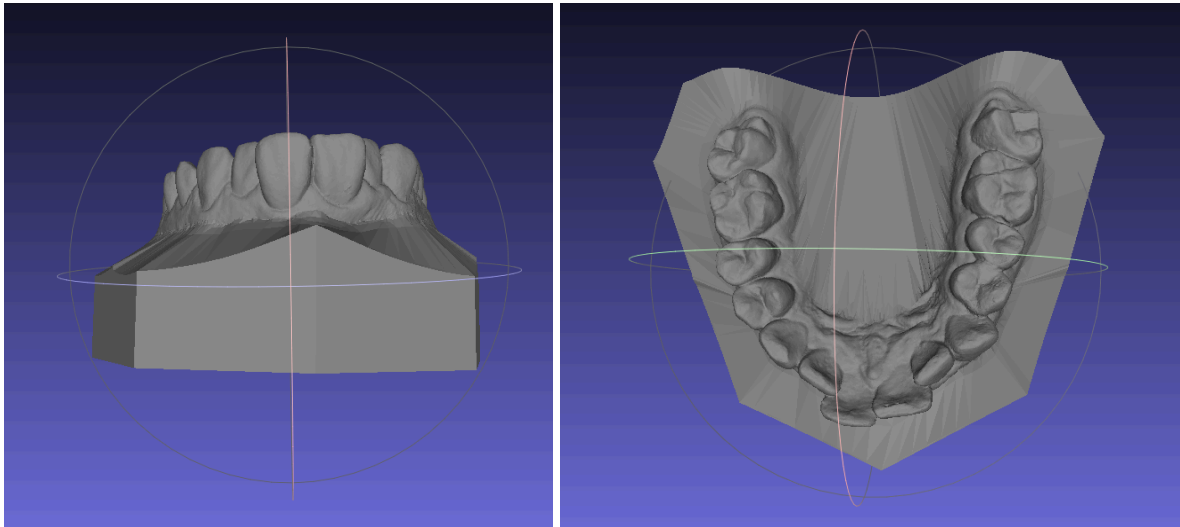


Figure 13: Original 01F4JV8X_upper.obj mesh scan

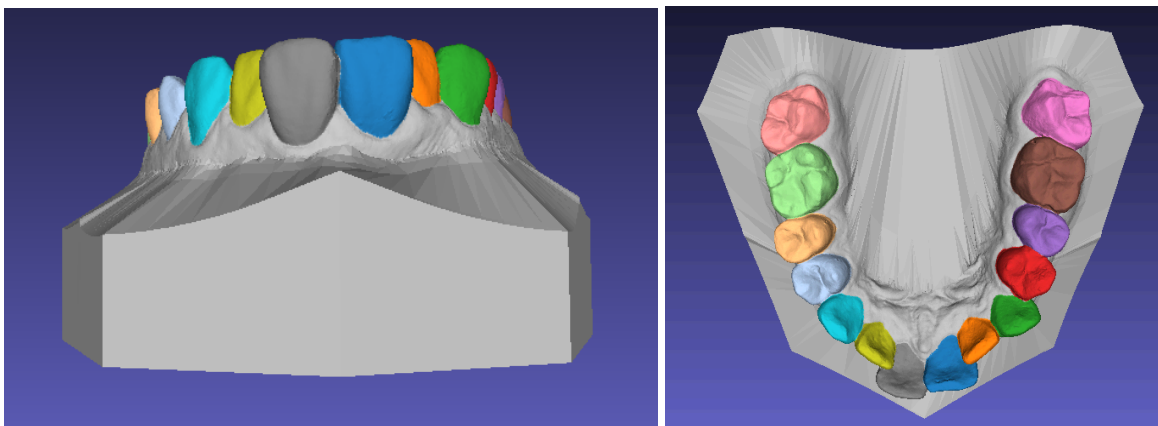


Figure 14: 01F4JV8X_upper.obj mesh scan with individual coloured teeth after segmentation

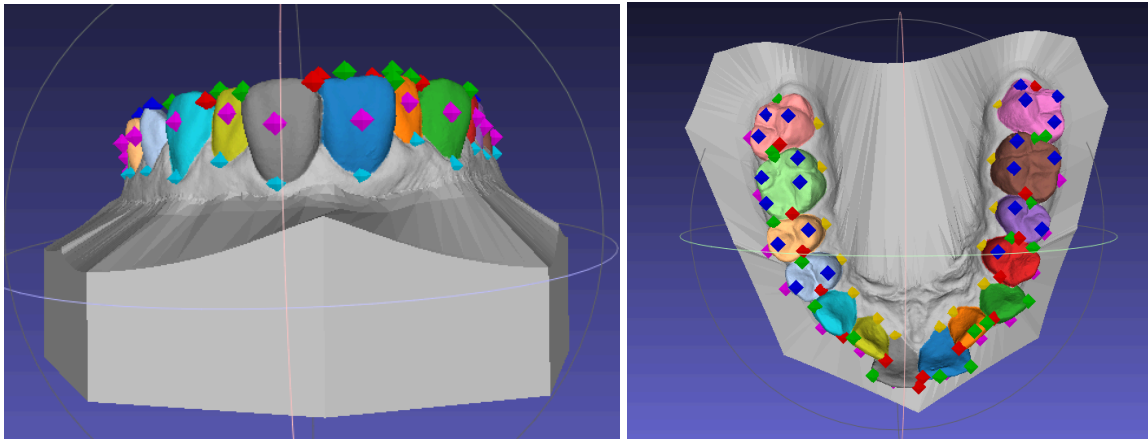


Figure 15: 01F4JV8X_upper_colored.ply with all landmark labels displayed on original 01F4JV8X_upper mesh

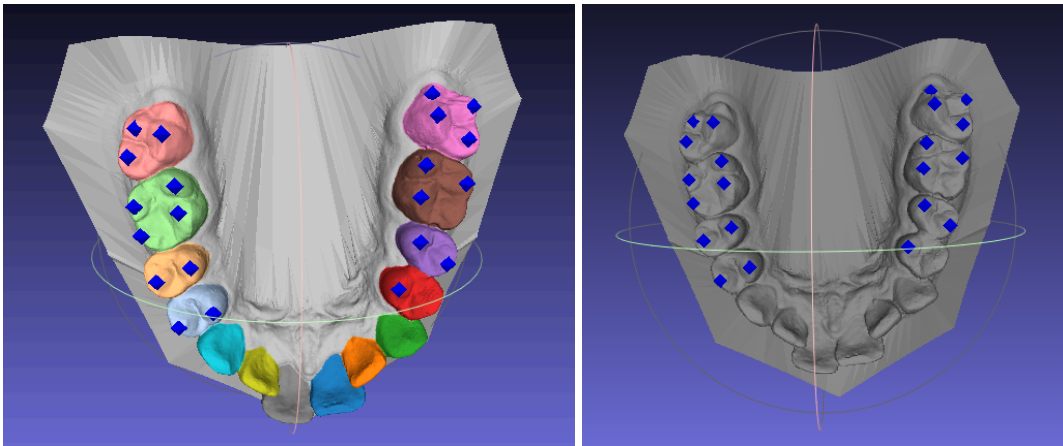


Figure 16: 01F4JV8X_upper_Cusp.ply displaying blue octahedron indicating Cusp landmarks on original 01F4JV8X_upper mesh

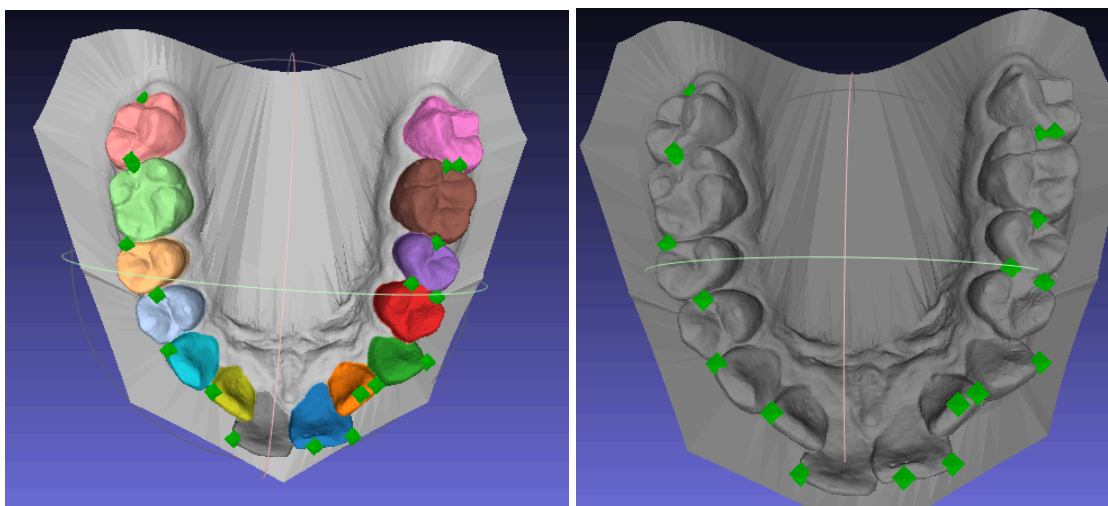


Figure 17: 01F4JV8X_upper_Distal.ply displaying green octahedron indicating Distal landmarks on original 01F4JV8X_upper mesh

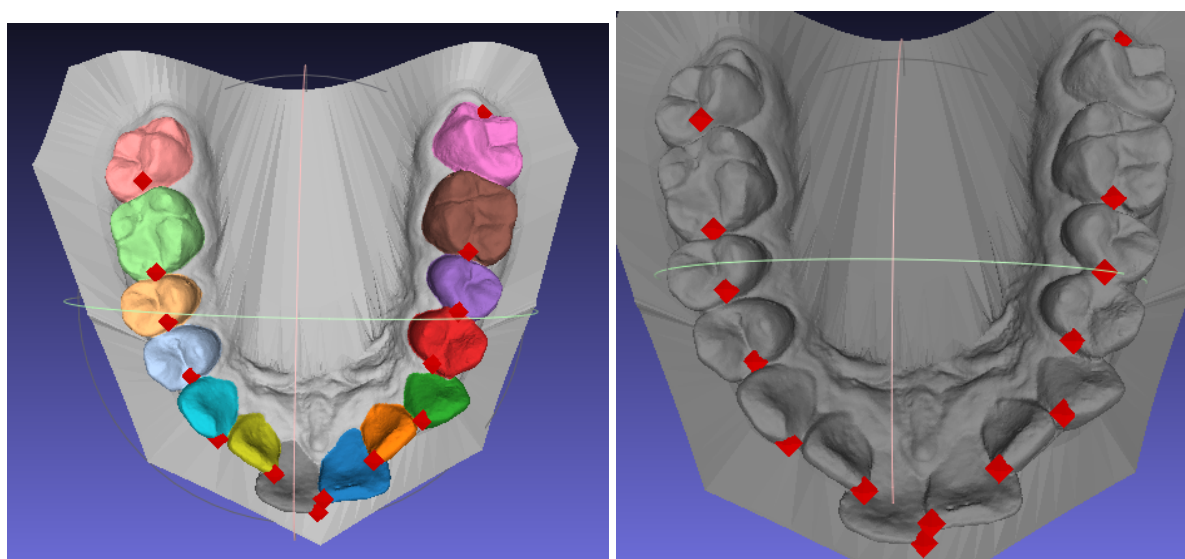


Figure 18: 01F4JV8X_upper_Mesial.ply displaying red octahedron indicating Mesial landmarks on original 01F4JV8X_upper mesh

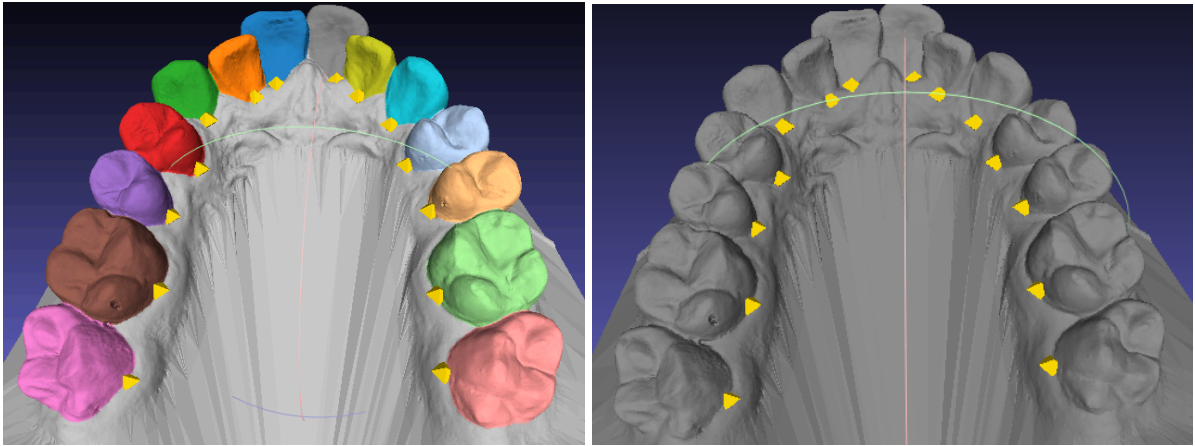


Figure 19: 01F4JV8X_upper_InnerPoint.ply displaying yellow octahedron indicating inner point landmarks on original 01F4JV8X_upper mesh

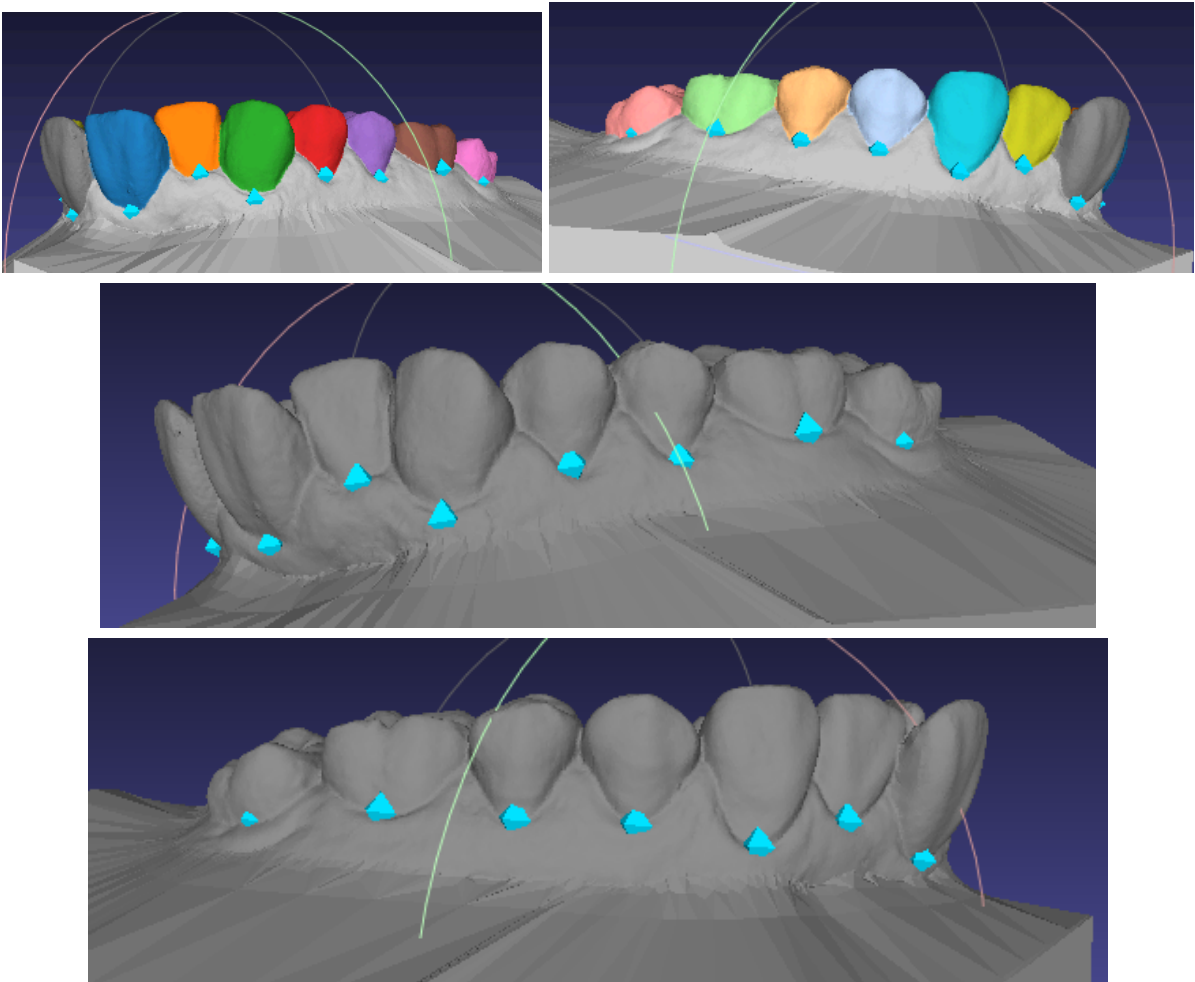


Figure 20: 01F4JV8X_upper_OuterPoint.ply displaying cyan octahedron indicating outer point landmarks on original 01F4JV8X_upper mesh

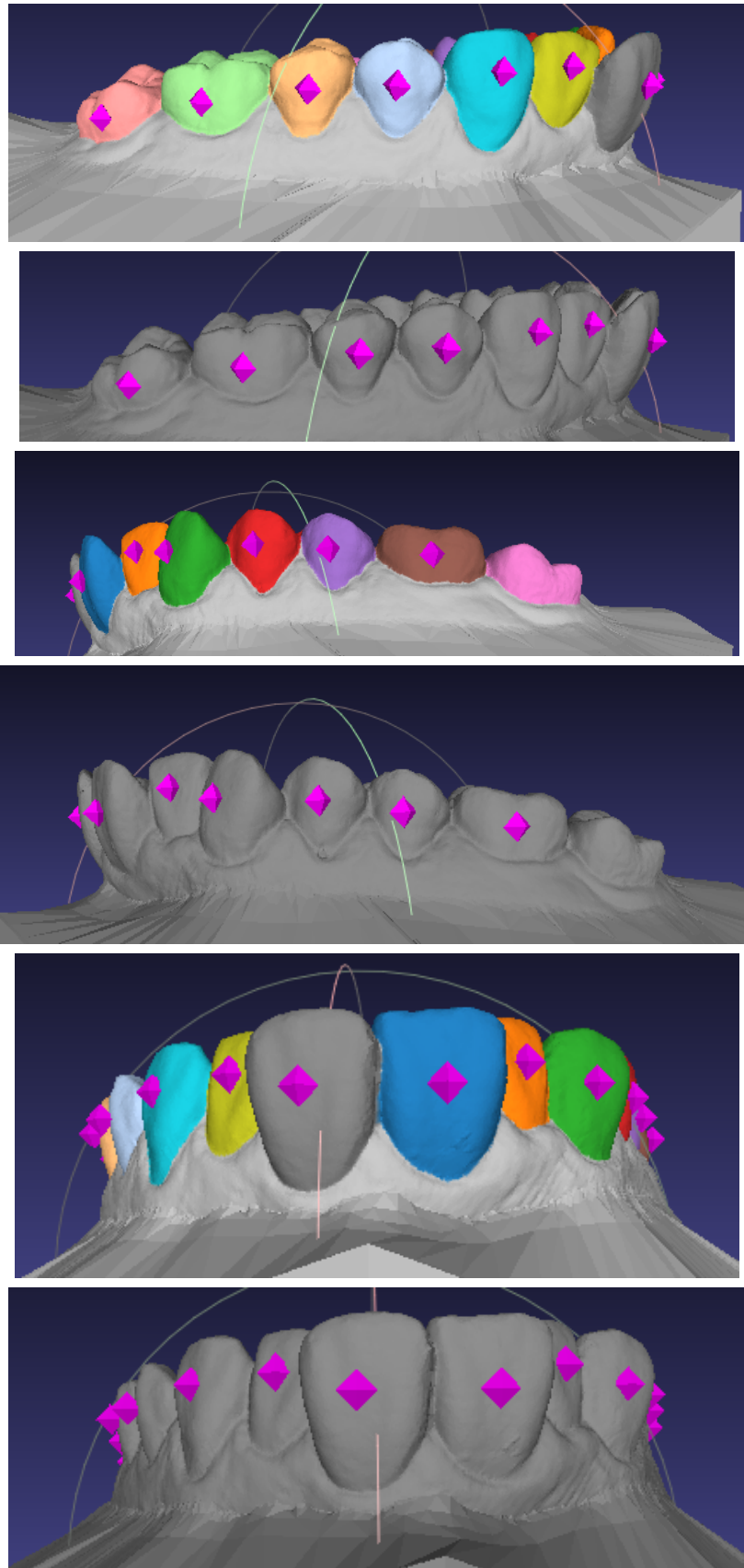


Figure 21: 01F4JV8X_upper_FacialPoint.ply displaying purple octahedron indicating facial point landmarks on original 01F4JV8X_upper mesh

5.6 Visualisation Files Output Discussion

The visualisation output files presented in Figures 13 to 20 provide a qualitative visual assessment of the pipeline's end-to-end performance on scan 01F4JV8X_upper. With the figures, the concurrent results provided from the teeth segmentation and landmark detection portion are shown in tandem, allowing evaluation on the task of segmentation and landmark detection.

Overall Segmentation Quality

Comparing the raw input mesh of Figure 13 against the segmented teeth output in Figure 14 confirms that the segmentation network has successfully isolated each tooth from one another and the gingival background. Each of the 14 segmented teeth are rendered to have defined colours to show clear distinction from one another. Moreover, the boundaries between adjacent teeth are well-defined to show the border that separates the teeth, and specific clearly the specific point in the mesh which corresponds to the right tooth, aligning to the TSA score of 0.9838 achieved by the pipeline. The gingival regions are correctly excluded from the coloured tooth instances and no obvious colour bleeding between neighbouring teeth are visible, suggesting that the Boundary Aware Deep Learning Network's refinement step has effectively resolved the inter-tooth boundary ambiguities at the contact regions.

Cusp Landmarks (Figure 16 - Blue Octahedra)

The blue octahedra are positioned on the occlusal surfaces of and at the incisal edges and canine tips of anterior teeth, which is consistent with the cusp landmark definition from Table 1. For this run of the pipeline, a total of 21 cusp markers were detected across the 14 teeth in the scan. The number of cusps detection from the pipeline aligns with the expected cusp count per teeth from Table 5, where we anticipated 0 cusps on incisors, 1 to 2 on premolars and 2 to 5 on molars.

Distal Landmarks (Figure 17 - Green Octahedra)

The detected distal markers are prominently visible on the proximal surfaces of teeth facing away from the arch midline, which aligns correctly with the anatomical definition of the distal contact point provided in Table 1. However, the count of 18 markers instead of an expected 14 markers with one for each teeth, shows that the pipeline may have some inconsistencies with detecting distal points as accurately as possible. Some teeth seemed to have duplicates on points that are furthest from the midline. Possible issues in the pipeline

which may have caused this to arise could be some the teeth marked with 2 or more distal landmarks had points that were equally maximal from the midline. A tighter neighbourhood radius or a post-clustering deduplication step that enforces a maximum of one distal detection per FDI tooth instance would likely resolve this issue.

Mesial Landmarks (Figure 18 - Green Octahedra)

The 14 mesial markers match the expected landmark count per tooth exactly, with exactly one mesial detection per tooth across the full arch. The positioning of the red octahedra on the midline-facing proximal surfaces of each tooth also prove to be correct according to anatomical definition. The lateral and top-down views both confirm that the mesial markers are consistently on the opposite proximal face to distal markers in Figure 16, demonstrating the mesial-distal prediction head successfully learned to discriminate between the surfaces.

Inner Point Landmarks (Figure 19 - Yellow Octahedra)

Once again, the 14 inner point markers are consistent with the expected number of one inner point per tooth, which are clearly distributed along the boundary of the inner mouth arch. The top-down view clearly shows this as the yellow octahedras trace a smooth-arch-shaped curve along the inner aspect of the dental arch from the molar of one end to another, which is exactly where the crown-gingiva boundary on the palatal side is expected to lie on.

Outer Point Landmarks (Figure 20 - Cyan Octahedra)

The outer point markers in Figure 19 are positioned on the buccal-facing (cheek-facing) surface of each tooth at the cervical boundary, which consistently aligns with the anatomical definition in Table 1. The cyan markers clearly sit at the lower convex edge of each visible teeth, and they run consistently along the gingival boundary along the outer surface of the mouth arch. Once again, the count of 14 cyan markers exactly matches the tooth count, and the uniform spacing of the cyan markers across the arch is consistent with expected positions of outer points.

Facial Point Landmarks (Figure 21 - Purple Octahedra)

The 13 facial point markers represent the only class where detection count falls below the expected count of 14 points, indicating a missed detection on the final molar at the back. Nevertheless, all the purple octahedra are correctly positioned on the direct front-facing surface of anterior teeth and on the cheek-facing surface of premolars and molars. The single missed detection on the posterior molar is consistent with a known limitation of the per-tooth

crop extraction in Stage 2. For teeth situated at the outermost extremes of the arch, the 12,000 point KDTree crop may have missed capturing the effective facial surface itself due to the molar with the missed facial point being positioned at the back of the dental arch.

Overall, the segmentation network of the pipeline performed very well, perfectly detecting all 14 individual teeth from the gingival background with well-defined inter-tooth boundaries and minimal colour blending at contact regions and boundaries. On the other hand, the landmark detection network demonstrated reliable performance across majority of landmark classes, with the mesial, inner point and outer point classes each achieving the expected count of exactly 14 detections with anatomically consistent placement across the arch. The distal class showed a moderate over-detection artefact while facial point class produced one missed detection at a posterior tooth, both of which were explainable due to the limitations of post-processing design rather than failures of the network itself. Cusp detection performed reliably despite worries of it being the greatest challenge to the pipeline due to its inconsistent counts and characteristics across different teeth classes. Taken together, the visualisation outputs confirm that the pipeline functions as a coherent end-to-end system, producing clinically interpretable landmark annotations from a raw intraoral scan without any manual intervention.

Chapter 6 Conclusion & Future Works

6.1 Conclusion

This final year project has delivered a complete, working pipeline for automated tooth segmentation and anatomical landmark detection from raw 3D intraoral scans. The two independently trained networks, a Point Transformer encoder-decoder segmentation network and a Stratified Transformer landmark detection network, are connected through a custom coordinate bridge that correctly handles the distinct coordinate systems and normalisation conventions between the two networks, enabling both networks to operate as a coherent end-to-end pipeline.

The teeth segmentation network employs a hierarchical Point Transformer architecture to perform semantic and instance segmentation of the full jaw mesh, assigning each vertex a standardised FDI tooth number. Meanwhile, the anatomical landmark detection network leverages Stratified Transformer with multi-scale attention windowed attention, operating at different resolutions to detect up to six anatomical landmarks per tooth, with per-point offset voting and DBSCAN-based clustering used to recover the final landmark positions from the dense network output. Evaluation on a held-out sample demonstrates strong segmentation boundary accuracy and competitive landmark detection performance, with FDI label identification and landmark recall identified as areas for future improvement..

The work demonstrates that the two distinct models, originally designed for separate tasks and trained under different data conventions, can be successfully integrated into a single cohesive pipeline through careful analysis and thorough planning on the overall pipeline configurations. The result is a fully automated system capable of producing clinically relevant dental landmark annotations from raw 3D intraoral scans without any manual intervention, with direct applicability to orthodontic treatment planning and digital workflow automation.

6.2 Future Works

Training dataset limitation. The Teeth3DS training dataset was heavily skewed towards patients undergoing active orthodontic or prosthetic treatment, with a demographic that is predominantly adolescent. Consequently, certain tooth classes would be notably underrepresented during training, notably wisdom teeth, which present atypical crown geometries that the network has limited exposure to. This imbalance likely contributed to the pipeline’s relatively low Teeth Identification Rate (TIR: 0.4794). Data augmentation techniques that explicitly oversample scans that contain underrepresented tooth classes could introduce a more balanced training distribution without requiring additional data collection. More broadly, expanding the training corpus to include intraoral scans from a wider and more demographically diverse patient population that encompasses healthy adult dentitions, elderly patients and non-orthodontic cases might improve the pipeline’s generalisation.

Extension to Full-Jaw Landmark Detection. The current landmark detection stage operates on individual per-tooth crops, which constrains the network to reason solely within the local geometry of a single tooth. An alternative formulation would be to process the full jaw simultaneously and capture inter-tooth spatial relationship which are unavailable at the per-tooth crop level.. Processing the full jaw simultaneously could potentially improve the spatial consistency of detected landmarks across the arch, reducing cases where per-tooth predictions are locally plausible but inconsistent with the patient’s overall global dental anatomy.

Clinical Validation. Validation of the pipeline results against landmark annotations produced independently by multiple practising orthodontists on unseen patient scans would provide a more clinically meaningful measure of the pipeline’s accuracy. This would establish whether detection error falls within the range of acceptable inter-annotator variability. Such a validation study would create a structured feedback mechanism, where systematic disagreements between the pipeline’s predictions and clinician annotations could be used to generate training examples, enabling targeted retraining on specific cases and tooth types where the pipeline underperforms.

References

- [1] A. Agarwal, “An Overview of Orthodontic Indices,” *World Journal of Dentistry*, vol. 3, no. 1, pp. 77–86, 2012, doi: <https://doi.org/10.5005/jp-journals-10015-1132>.
- [2] Y. Koizumi et al., “Maxillofacial Morphology as a Predictive Factor for Caries Risk in Orthodontic Patients: A Cross-Sectional Study,” *Journal of Clinical Medicine*, vol. 13, no. 2, pp. 622–622, Jan. 2024, doi: <https://doi.org/10.3390/jcm13020622>.
- [3] F. Schwendicke, C. E. Dörfer, P. Schlattmann, L. Foster Page, W. M. Thomson, and S. Paris, “Socioeconomic inequality and caries: a systematic review and meta-analysis,” *Journal of Dental Research*, vol. 94, no. 1, pp. 10–18, Jan. 2015, doi: <https://doi.org/10.1177/0022034514557546>.
- [4] [1]J. Liu et al., “Deep learning-enabled 3D multimodal fusion of cone-beam CT and intraoral mesh scans for clinically applicable tooth-bone reconstruction,” *Patterns*, vol. 4, no. 9, p. 100825, Sep. 2023, doi: <https://doi.org/10.1016/j.patter.2023.100825>.
- [5] S. Xi, Z. Liu, J. Chang, H. Wu, X. Wang, and A. Hao, “3D Dental Model Segmentation with Geometrical Boundary Preserving,” *arXiv.org*, 2025. <https://arxiv.org/abs/2503.23702>.
- [6] Y. Cai, Y. Xu, Z. Hu, J. Chen, and J. Huang, “BATISNet: Instance Segmentation of Tooth Point Clouds with Boundary Awareness,” *arXiv.org*, 2025. <https://arxiv.org/abs/2512.24201>.
- [7] T.-H. Wu et al., “Two-Stage Mesh Deep Learning for Automated Tooth Segmentation and Landmark Localization on 3D Intraoral Scans,” *IEEE Transactions on Medical Imaging*, vol. 41, no. 11, pp. 3158–3166, Nov. 2022, doi: <https://doi.org/10.1109/tmi.2022.3180343>.
- [8] C. D. A. Kathryn Truant, “The International Tooth Numbering System (FDI),” *pressbooks.pub*, Aug. 2022, Available: <https://pressbooks.pub/oralsurgery4rns/part/the-international-tooth-numbering-system/>
- [9] H. Smith, “Fédération Dentaire Internationale (FDI) notation | Radiology Reference Article | Radiopaedia.org,” *Radiopaedia*. <https://radiopaedia.org/articles/federation-dentaire-internationale-fdi-notation>
- [10] A. Ben-Hamadou et al., “3DTeethSeg’22: 3D Teeth Scan Segmentation and Labeling Challenge,” *arXiv.org*, 2023. <https://arxiv.org/abs/2305.18277>

- [11] “3D Teeth Scan Segmentation and Labeling Challenge MICCAI2022 - Grand Challenge,” *grand-challenge.org*, 2022. <https://3dteethseg.grand-challenge.org/>
- [12] A. Ben-Hamadou *et al.*, “3DTeethSeg’22: 3D Teeth Scan Segmentation and Labeling Challenge,” *arXiv.org*, 2023. <https://arxiv.org/abs/2305.18277>.
- [13] A. Ben-Hamadou *et al.*, “Teeth3DS+: An extended benchmark for intraoral 3D scans analysis,” Teeth3DS+: An Extended Benchmark for Intraoral 3D Scans Analysis, <https://arxiv.org/pdf/2210.06094>.
- [14] A. Ben-Hamadou *et al.*, “Detecting Dental Landmarks from Intraoral 3D Scans: the 3DTeethLand challenge,” *arXiv.org*, 2025. <https://arxiv.org/abs/2512.08323>.
- [15] S. Bionetworks, “3DTeethLand-MICCAI2024,” *Synapse.org*, 2024. <https://www.synapse.org/Synapse:syn57400900/wiki/627259>.
- [16] T. Kubík *et al.*, “LMVSegRNN and Poseidon3D: Addressing challenging teeth segmentation cases in 3D dental surface orthodontic scans,” *Bioengineering*, vol. 11, no. 10, p. 1014, Oct. 2024.
- [17] A. Ben-Hamadou *et al.*, “Teeth3DS: a benchmark for teeth segmentation and labeling from intra-oral 3D scans,” *arXiv (Cornell University)*, Jan. 2022, doi: <https://doi.org/10.48550/arxiv.2210.06094>.
- [18] A. Rekik *et al.*, “TSegLab: Multi-stage 3D dental scan segmentation and labeling,” *Comput. Biol. Med.*, vol. 183, p. 109300, Dec. 2024.
- [19] A. Ben-Hamadou *et al.*, “Detecting dental landmarks from intraoral 3D scans: the 3DTeethLand challenge,” *arXiv preprint arXiv:2512.08323*, Dec. 2025.
- [20] N. van Nistelrooij and S. Vinayahalingam, “ToothInstanceNet: Comprehensive Information from Intra-oral Scans by Integration of Large-Context and High-Resolution Predictions,” *Lecture Notes in Computer Science*, pp. 229–240, 2025, doi: https://doi.org/10.1007/978-3-031-88977-6_21.
- [21] F. Eggmann and M. B. Blatz, “Recent Advances in Intraoral Scanners,” *Journal of Dental Research*, vol. 103, no. 13, Oct. 2024, doi: <https://doi.org/10.1177/00220345241271937>.

- [22] Ling, Z. (2024). Semi-supervised tooth instance segmentation. Final Year Project (FYP), Nanyang Technological University, Singapore. <https://hdl.handle.net/10356/176834>
- [23] B. Woodsend, E. Koufoudaki, P. A. Mossey, and P. Lin, "Automatic recognition of landmarks on digital dental models" *Comput. Biol. Med.*, vol. 137, p. 104819, 2021.
- [24] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE CVPR*, 2017, pp. 652–660.
- [25] C. R. Qi, L. Yi, H. Su, and Guibas, Leonidas J, "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," *arXiv.org*, 2017. <https://arxiv.org/abs/1706.02413>
- [26] H. Zhao, L. Jiang, J. Jia, P. H. Torr, and V. Koltun, "Point transformer," in *Proc. IEEE ICCV*, 2021, pp. 16259–16268.
- [27] X. Lai, J. Liu, L. Jiang, L. Wang, H. Zhao, S. Liu, X. Qi, and J. Jia, "Stratified transformer for 3D point cloud segmentation," in *Proc. IEEE CVPR*, 2022, pp. 8500–8509.

Appendix

Hyperparameters

1. Farthest Point Sampling Segmentation Parameters

```
_FPS_MODEL_PARAMS = {  
    "input_feat": 6,  
    "stride":     [1, 4, 4, 4, 4],  
    "nstride":    [2, 2, 2, 2],  
    "nsample":    [36, 24, 24, 24, 24],  
    "blocks":     [2, 3, 4, 6, 3],  
    "block_num":  5,  
    "planes":     [32, 64, 128, 256, 512],  
    "crop_sample_size": 3072,  
}
```

Hyperparameter	Value	Meaning
stride	[1, 4, 4, 4, 4]	Point reduction factor at each encoder level
nsample	[36, 24, 24, 24, 24]	Local neighbourhood size at each level
blocks	[2, 3, 4, 6, 3]	Transformer blocks per encoder level
block_num	5	Number of encoder levels
planes	[32, 64, 128, 256, 512]	Feature dimension at each level
crop_sample_size	3072	Points per local crop window
Output classes (k)	10	9 tooth and 1 gingiva labels

2. Boundary Aware Deep Learning Network

```
_BDL_MODEL_PARAMS = {
    "input_feat": 6,
    "stride": [1, 1],
    "nsample": [36, 24],
    "blocks": [2, 3],
    "block_num": 2,
    "planes": [16, 32],
    "crop_sample_size": 3072,
}
```

Hyperparameter	Value	Meaning
stride	[1, 1]	No downsampling, all boundary points are kept
nsample	[36, 24]	Neighbourhood sizes for 2 levels
blocks	[2, 3]	Transformer blocks per level
block_num	2	Lighter 2-level encoder
planes	[16, 32]	Narrower feature dimensions
crop_sample_size	3072	Same local crop size
Output classes (k)	2	Binary: foreground tooth vs. background

3. Boundary Sampling Parameters

```
"boundary_sampling_info": {
    "bdl_ratio": 0.7,
    "num_of_bdl_points": 20000,
    "num_of_all_points": 24000,
},
```

Hyperparameter	Value	Meaning
bdl_ratio	0.7	Points where majority-label fraction < 70% are boundary points
num_of_bdl_points	20,000	Max boundary points to resample
num_of_all_points	24,000	Total points fed to Module 2
KNN neighbours for boundary detection	k=40	inference_pipeline.py:306

4. KDTree Hyperparameters

Hyperparameter	Value	Meaning
leaf_size for Teeth Segmentation	2	Controls how many points are stored in each leaf node of KDTree
k (nearest neighbours)	1	For each boundary point, only the closest point in the sampled prediction is stored.
leaf_size for Landmark Detection Crop Extraction	16	Controls how many points are stored in each leaf node of KDTree
k (crop extraction)	12,000 (default)	12,000 nearest vertices in the z-score normalised space to the tooth centroid are selected as the input crop.

5. KPConv Point Embedding

```
point_embedding:  
  use: True  
  kpconv_point_influence: 0.02  
  kpconv_ball_radius: 0.05
```

Hyperparameter	Value	Meaning
kpconv_point_influence	0.02	≈ 0.35 mm — kernel point influence radius
kpconv_ball_radius	0.05	≈ 0.87 mm — ball query radius for neighbourhood

6. Stratified Transformer

```
channels_list: [48, 96, 192, 256]  
out_channels: [1, 4, 4, 4, 4, 4]  
depths: [3, 9, 3]  
heads_list: [6, 12, 24]  
window_sizes: [0.1, 0.2, 0.4]  
point_embedding:  
  use: True  
  kpconv_point_influence: 0.02  
  kpconv_ball_radius: 0.05  
stratified_union: False  
downsample_ratio: 0.26  
max_drop_path_prob: 0.3  
stratified_downsample_ratio: 0.26
```

```
crpe_bins: 80
transformer_lr_ratio: 0.1
```

Hyperparameter	Value	Meaning
channels_list	[48, 96, 192, 256]	Feature width after embedding and at each encoder scale
out_channels	[1, 4, 4, 4, 4, 4]	Output channels per head (seg head and 5 landmark heads)
depths	[3, 9, 3]	Transformer blocks per encoder scale
heads_list	[6, 12, 24]	Attention heads at each scale
window_sizes	[0.1, 0.2, 0.4]	
downsample_ratio	0.26	FPS ratio per downsample step (26%)
stratified_downsample_ratio	0.26	FPS ratio for sparse key selection inside attention
max_drop_path_prob	0.3	Maximum stochastic depth probability (regularisation)
stratified_union	False	Use DenseOrSparse (not DenseAndSparse) attention variant
transformer_lr_ratio	0.1	Transformer weights trained at 10% of base LR
crpe_bins	80	Bins per axis for position encoding tables

7. DBSCAN Landmark Clustering

```
dbscan_cfg:
  max_neighbor_dist: 0.03
  min_points: 40 # [30, 50]
  weighted_cluster: True
  weighted_average: True
```

Hyperparameter	Value	Physical meaning
dist_thresh	0.12 (norm units)	Physical points beyond this distance are discarded during voting
max_neighbor_dist	0.03 (norm units)	Physical point within this range form one cluster for DBSCAN
min_points	20 was instantiated in pipeline	Minimum votes for a valid cluster

weighted_cluster	True	Confidence weights influence cluster membership
weighted_average	True	Centroid = weighted mean of member votes

8. Training Hyperparameters

Hyperparameter	Value
lr (base learning rate)	0.0006
weight_decay	0.0001
epochs	500
warmup_epochs	5
Optimiser	AdamW
LR schedule	LinearWarmup to CosineAnnealing
accumulate_grad_batches	1
gradient_clip_norm	35
batch_size	16
proposal_points (crop size)	12,000